LLM ENGINEERING

Week 02

Building a Transformer Block

Thursday, 22 October 2026

Interactive version: https://llm.strueber.dev/#/week-02

Bethgelab · University of Tübingen · Winter term 2026/27

Text and figures under CC BY 4.0, code under the MIT licence.

Recap & Roadmap

1

Announcements

Next week, 29 October

- Lecture as a video, not in person
- Tutorials as usual

Quiz 1 on ILIAS

- Weeks 1, 2 and Bonus 1
- Due Friday 30 October, end of day

First exercise: this lecture

- Due Wednesday 28 October, 22:00
- Email me: partner and GitHub names

▼ Dates for the coming week

Next week's lecture, on 29 October, does not take place in person. It comes as a video, and the tutorials run as usual.

Quiz 1 is on ILIAS. You can take it whenever you are ready, up to the end of Friday, 30 October, and you will get a reminder by email. It covers Week 1, this lecture and Bonus 1.

The first coding exercise covers the content of this lecture and is due on Wednesday, 28 October, at 22:00. Email the lecturer the name of your exercise partner and the GitHub account names of both of you. We then invite you to a repository of your own, in which you work on the assignments. The automated tests run there, which is why we need your accounts.

You have the objective, not yet the function

$$P(x_1, \ldots, x_n) = \prod_{t=1}^{n} P(x_t \mid x_{<t}) \tag{1}$$

- Raw window: `[B,S+1]` token IDs
- Shift once: inputs, labels `[B,S]`
- Wanted: logits `[B,S,V]`

▼ **From the objective to one training batch**

Last week you defined the training objective: predict each token from the tokens before it. Formula (1) factors the probability of any sequence into exactly these predictions, so every token after the first is the target for the prefix that precedes it.

In code this takes one shift. Write B for the number of sequences in a batch, S for the sequence length after the shift, D for the model width and V for the vocabulary size. A batch arrives as integer token IDs `raw` of shape `[B,S+1]`, and

```
inputs = raw[:, :-1]   # [B,S]
labels = raw[:, 1:]    # [B,S]
```

The model we need is a function from `inputs` to one probability distribution per position. It first computes hidden states of shape `[B,S,D]` and then logits of shape `[B,S,V]`. Logit row i is scored against label i, and label i is raw token $i + 1$, so every position predicts the token that follows it.

The function we build today is a decoder-only Transformer. It reads a sequence of token IDs and predicts the next token at every position.

3

How does a model predict the next token, and at what cost?

- **Attention**: look back, causally
- Block: residuals, normalization, multilayer perceptron (MLP)
- Cost: parameters, FLOPs, memory
- Exercise: your first trained language model

▼ Today's question

The question for this lecture is: how does a model turn the context into a next-token prediction, and what does that cost? It has three parts.

The first part is attention, the mechanism that lets each position look back at the earlier tokens, and only at those. The second is the block that wraps attention: residual connections, normalization and a multilayer perceptron (MLP) that transforms each position on its own. The third is the cost of the assembled model, counted as parameters, floating-point operations (FLOPs) and memory.

The exercise puts the three parts together. You build one attention head, one block and a first training loop, train your first language model, and check your cost estimates against measurements.

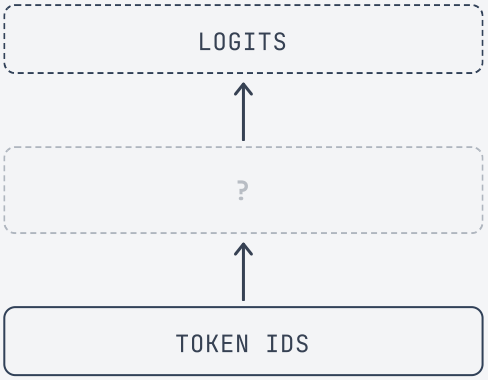
4

What must a function do?

Input: the token IDs of “The cat sat on the”
Output: a distribution over the next token

Today's build, one diagram

- Token embeddings
- Position signal
- Causal self-attention
- MLP
- Output head



▼ **Five components, one block**

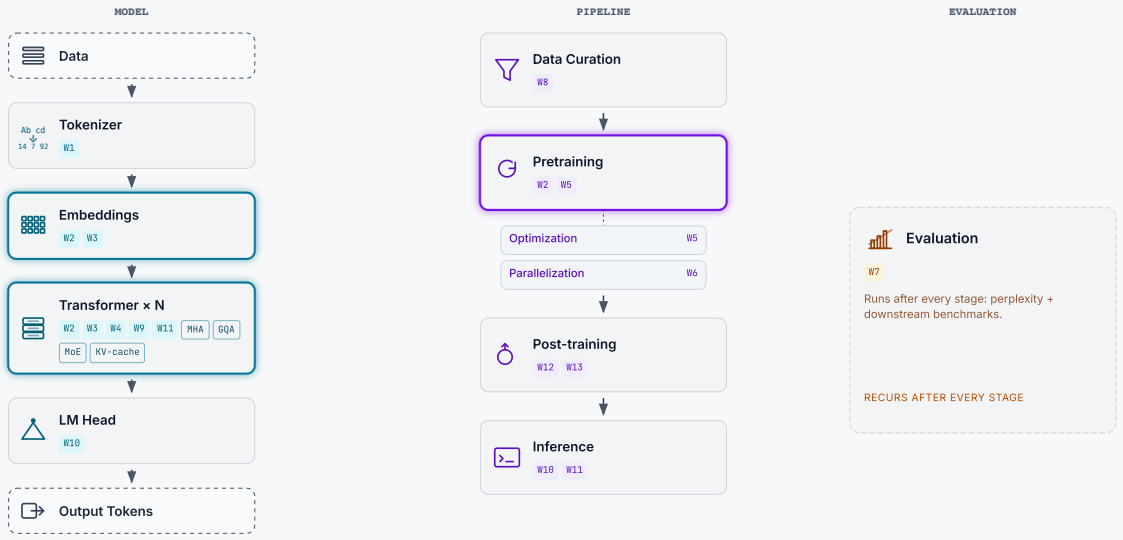
A function that turns token IDs into a next-token distribution has to do five things. It must turn each ID into numbers it can compute with, know where each token stands, look back at the relevant earlier tokens, transform what it found, and score every vocabulary entry. The decoder-only Transformer has one component for each: token embeddings, a position signal, causal self-attention, an MLP and an output head.

The diagram starts almost empty. Token IDs flow into a single block that is not filled in yet, and on to the model output. The main path stays fixed while the components appear one at a time. Later slides add the residual paths and the normalization, and the output head and the loss come last, in the Minimal GPT tab. The diagram shows one block. Week 3 repeats the block to build depth and splits attention into several heads.

Where this fits

Read on your own

- Today: embeddings, one block, pretraining
- Week 3: heads, depth, RoPE, masks



10 CONNECTIONS

◀ Prev | Reset | Start ▶

Click "Start" to walk through how decisions in one part of the pipeline shape every other part.

▼ Reading the Week 2 pipeline

The highlighted stages are the ones this week works on: embeddings and the Transformer as the model, and pretraining as the training stage. Tokenization comes from Week 1. This lecture starts from token IDs and does not depend on how the tokenizer was trained.

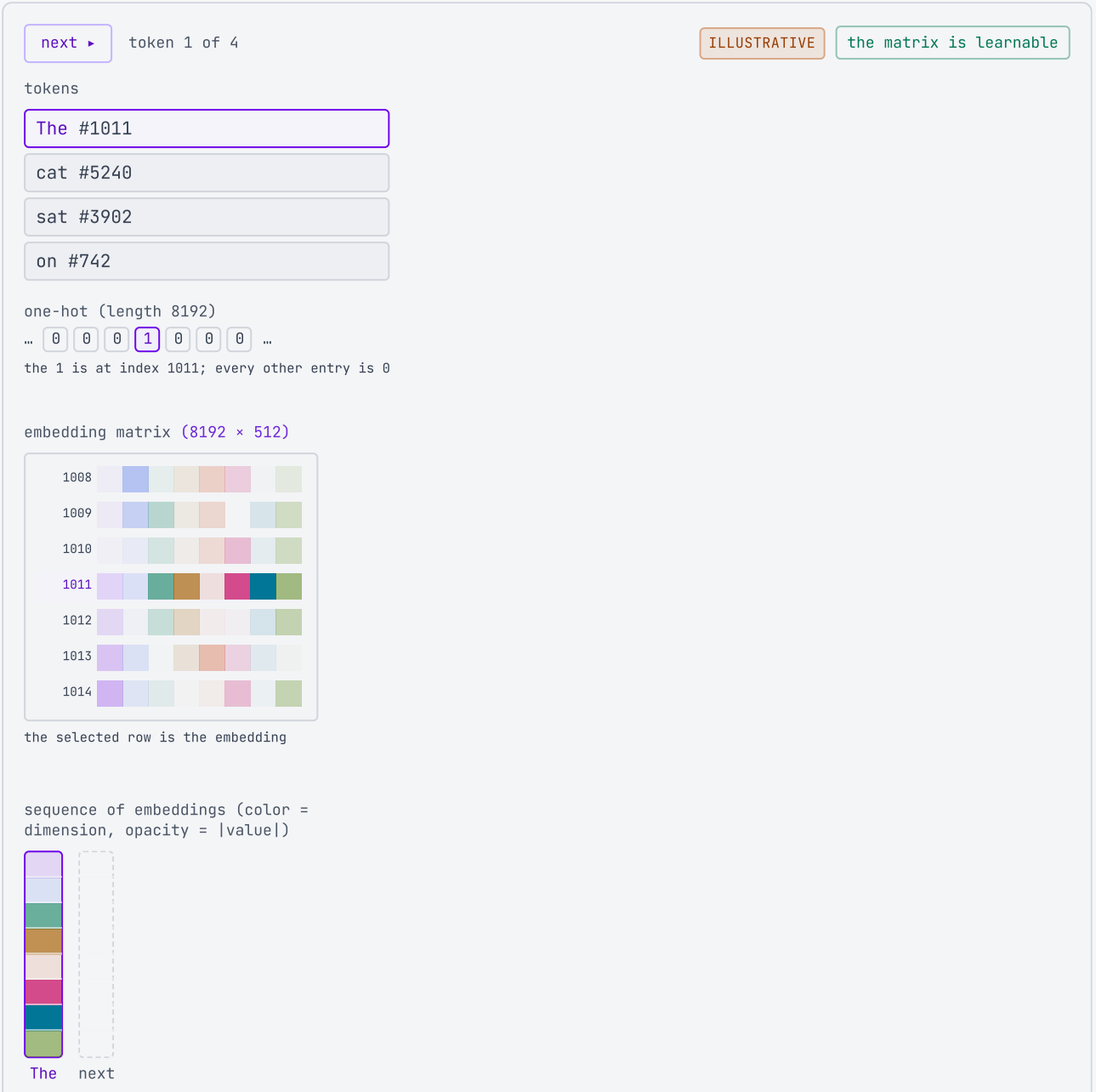
Today's model has token embeddings, one attention head, one pre-norm block, a sinusoidal position signal and a tied output head. Week 3 extends the attention and the block with multiple heads, a stack of repeated blocks, rotary position embedding (RoPE) and more detailed masks. The remaining stages belong to later weeks: optimization, parallelism, post-training, inference and evaluation.

Token Embeddings

An embedding table turns IDs into vectors

(2) $$E \in \mathbb{R}^{V \times D}, \qquad [B, S] \longrightarrow [B, S, D]$$

- IDs are addresses, not magnitudes
- `[B,S]` selects rows: `[B,S,D]`
- `VD = 8192·512 = 4,194,304` parameters



Choose next and find the highlighted token ID. Follow the 1 in its one-hot vector to the highlighted row of the matrix. That row is the token's embedding, and it appears again as the newest column of the output sequence. The widget draws only 8 of the course width's D = 512 dimensions and a small window of rows. The token IDs and cell values are illustrative, not learned embeddings.

▼ An ID selects one row of the table

Token IDs are category labels. ID 17 is no closer to ID 18 than to ID 4,000 in any sense that matters for language, so the model cannot compute with the IDs as numbers. It needs a learned vector for every token, and an embedding table provides one: a matrix $E \in \mathbb{R}^{V \times D}$ with one row per vocabulary entry.

An ID is the address of a row. For integer IDs of shape `[B,S]`, every ID selects one row of length D, so the lookup returns a tensor of shape `[B,S,D]`. Mathematically, the lookup of token t is the product of the one-hot vector $e_t \in \mathbb{R}^V$ with the table,

$$e_t^\top E = E_{t,:},$$

but an implementation fetches the row directly and never builds the one-hot vector. Because the lookup depends on the ID alone, a token that appears twice in a sequence receives the same vector at both positions. The position signal added next, and attention after it, make the two differ.

The table holds VD trainable parameters, which is $8192 \cdot 512 = 4{,}194{,}304$ for the course vocabulary and width. Every component of the block reads and writes vectors of the model width D.

Training arranges the rows so that the predictions improve. The objective does not require that tokens you would call similar end up close together.

next ▸ token 4 of 4 (wraps to start)

ILLUSTRATIVE the matrix is learnable

tokens

The #1011

cat #5240

sat #3902

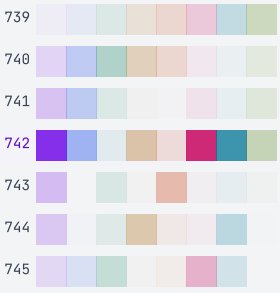
on #742

one-hot (length 8192)

… 0 0 0 1 0 0 0 …

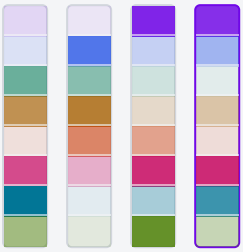
the 1 is at index 742; every other entry is 0

embedding matrix (8192 × 512)



the selected row is the embedding

sequence of embeddings (color = dimension, opacity = |value|)

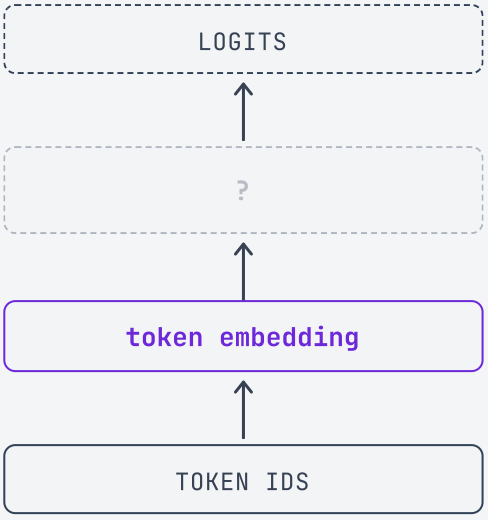


Choose next and find the highlighted token ID. Follow the 1 in its one-hot vector to the highlighted row of the matrix. That row is the token's embedding, and it appears again as the newest column of the output sequence. The widget draws only 8 of the course width's D = 512 dimensions and a small window of rows. The token IDs and cell values are illustrative, not learned embeddings.

The fourth token, "on": its ID selects one more row, and the output now holds one embedding column for each token of "The cat sat on".

Add embeddings to the model

- Each ID becomes one `D`-wide vector
- The same table returns as the output head



▼ **The lookup adds a width axis**

The embedding is the first component in the diagram. The lookup replaces each token ID with one vector of length D, so IDs of shape `[B,S]` become states of shape `[B,S,D]`. The batch and sequence axes stay as they are.

Follow the D-wide path onward. The position signal is added at the same width, and every branch of the block must return to that width. The same table comes back at the end as the tied output head, which reuses it without changing the input lookup.

Self-Attention

9

Attention is a soft, learned lookup

- Each position emits a **query**
- Every position offers a **key** and **value**
- Query scores keys, blends values

▼ Queries, keys and values are learned roles

The next token depends on the earlier context, and which earlier tokens matter depends on that context too. The model must learn which earlier positions matter to each current position, and how strongly to weight each of them.

Attention does this with a lookup. Each position emits a **query**, and every position offers a **key** and a **value**. The query is scored against every key, and the scores decide how much of each value goes into the result. Every step is differentiable, so attention is a soft lookup whose result depends on the context, and gradient descent can train it.

For an input state x_i, three learned projections produce the query $q_i = x_i W_Q$, the key $k_i = x_i W_K$ and the value $v_i = x_i W_V$. The query belongs to the position doing the lookup. The keys are what the query is compared with, and the values carry the information that gets mixed. All three come from the same input states. They are roles the model learns, with no fixed linguistic meaning.

A query-key dot product only sets a weight: the key vector itself never enters the output. After a softmax across the keys, the weights blend the value vectors.

Scaled dot-product attention

Projection	Weights	Result
`Q=XW_Q`	`W_Q:[D,d_k]`	`Q:[B,S,d_k]`
`K=XW_K`	`W_K:[D,d_k]`	`K:[B,S,d_k]`
`V=XW_V`	`W_V:[D,d_k]`	`V:[B,S,d_k]`

(3)
$$\mathrm{Attention}(Q,K,V) = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right)V$$

output projection
$$O = HW_O, \qquad W_O : [d_k, D], \qquad O : [B,S,D]$$

- Scores, probabilities: `[B,S,S]`
- Softmax across keys, per query
- Course config: one head, `d_k = D`

▼ From shapes to one attention row

Formula (3) defines scaled dot-product attention for one head of width d_k. The input states of a batch form a tensor X of shape `[B,S,D]`. Three weight matrices $W_Q, W_K, W_V \in \mathbb{R}^{D\times d_k}$ project them to the queries, keys and values Q, K and V, each of shape `[B,S,d_k]`. The matrix M is an additive mask. For causal attention, $M_{ij} = 0$ when key j is at or before query i, and $M_{ij} = -\infty$ otherwise. The two slides on the causal mask explain why.

One query at a time

Take one sequence and follow a single query $q_i \in \mathbb{R}^{d_k}$. Its scores against the keys $k_1, \ldots, k_S$, its attention weights and its output are

$$z_{ij} = \frac{q_i \cdot k_j}{\sqrt{d_k}} + M_{ij}, \qquad j = 1, \ldots, S,$$
$$a_{ij} = \frac{e^{z_{ij}}}{\sum_{l=1}^{S} e^{z_{il}}},$$
$$h_i = \sum_{j=1}^{S} a_{ij}\, v_j \ \in \mathbb{R}^{d_k}.$$

The implementation follows the same order: compute `q_i @ K.T`, divide by $\sqrt{d_k}$, add the mask, apply the softmax across the keys, and take the weighted sum of the value rows. Batched over every query, the scores and the probabilities are tensors of shape `[B,S,S]`, with one row per query, and every probability row sums to one. The value blends form H of shape `[B,S,d_k]`, and the output projection $W_O \in \mathbb{R}^{d_k \times D}$ maps them to O of shape `[B,S,D]`.

The Week 2 model has one head as wide as the model, $d_k = D$. The shapes are written for a general d_k because Week 3 splits the width into several narrower heads.

A row computed by hand

Take $d_k = 2$, three positions numbered 0, 1 and 2, and the query of position 1. The vectors are chosen by hand:

$$q_1 = (1,\ 1), \qquad K = \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{pmatrix}, \qquad V = \begin{pmatrix} 1 & 0 \\ 0 & 2 \\ 3 & 3 \end{pmatrix},$$

where row j of K and V belongs to position j. The scaled scores are $q_1 K^\top / \sqrt{2} = (1,\ 1,\ 2)/\sqrt{2}$. Position 2 lies in the future of position 1, so the mask row is $(0,\ 0,\ -\infty)$ and removes it. The two remaining scores are equal, so the softmax splits the weight evenly, $a_{1,:} = (0.5,\ 0.5,\ 0)$, and the output is

$$h_1 = 0.5\,(1,\ 0) + 0.5\,(0,\ 2) = (0.5,\ 1).$$

Without the mask, position 2 would take about half of the weight, because its score is the largest.

Scale first, then mask

Formula (3) scales the scores before it adds the mask. For the exact mask of 0 and $-\infty$ the order makes no difference, since $-\infty/\sqrt{d_k} = -\infty$. It matters for a finite bias, or for a large finite number used in place of $-\infty$: dividing either by $\sqrt{d_k}$ would shrink it.

Will rows get sharper or flatter?

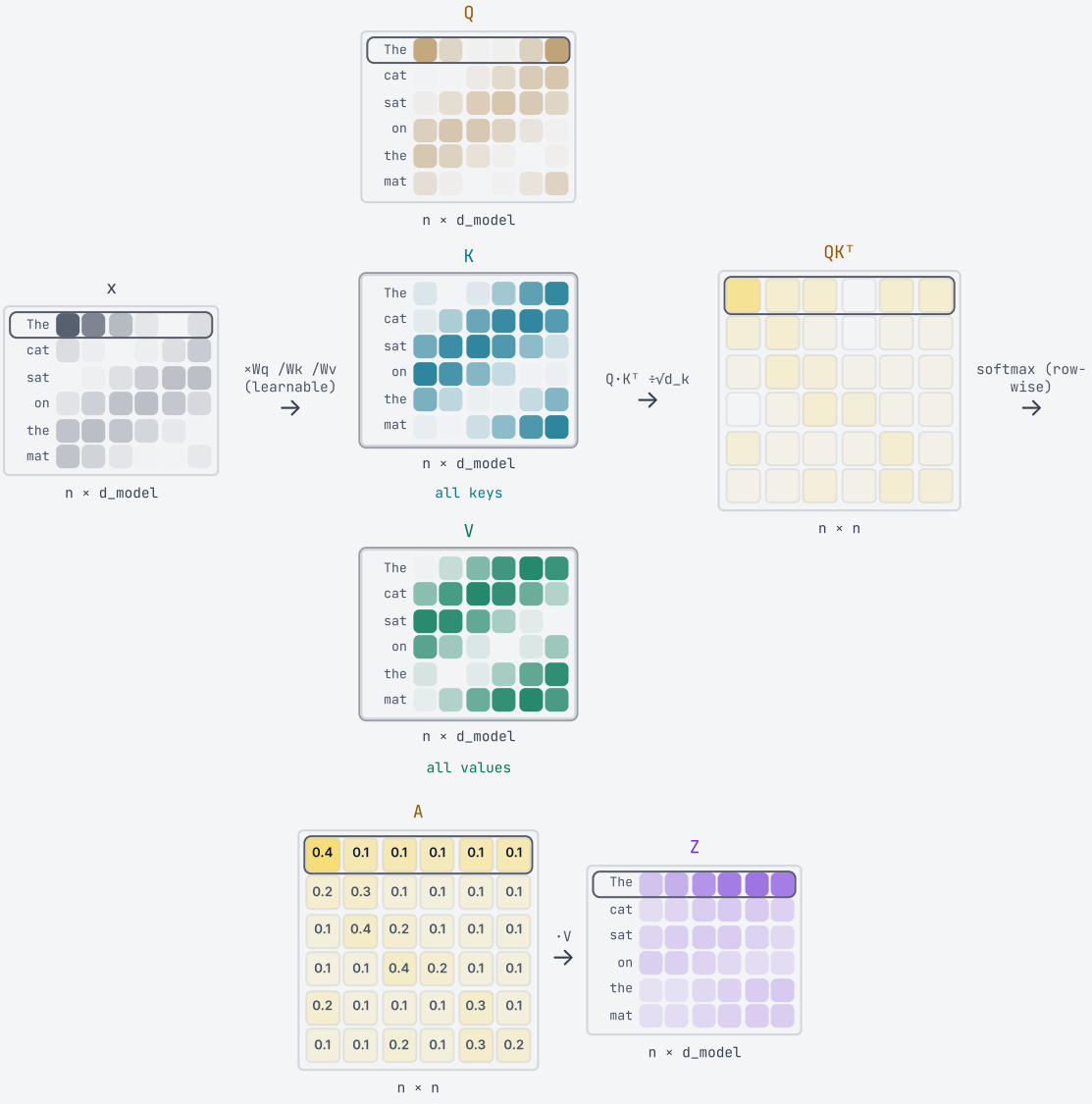
Turn off the √d_k scaling.

trace token: The | cat | sat | on | the | mat

options: scaling ON (÷√d_k, d_k = 16) | causal mask OFF

ILLUSTRATIVE Rows index the sequence (n tokens); columns index the model dimension d_model = D. The scores are toy values, scaled with a toy d_k = 16.

The tokens, channel fills, six columns and scores are synthetic, not learned or measured.



Read left to right: Z = softmax(QKᵀ ÷ √d_k) · V. The trace follows the token “The”. Its row in x produces one row of Q, the query. The query is compared with every key and blends every value, so K and V stay fully lit, while QKᵀ, A and Z each light up the query’s own row. Scaling multiplies every score by 1/√d_k, so the QKᵀ panel dims when you turn it on. The row-wise softmax then turns the scores into the weights A. Toggle scaling and the causal mask one at a time: scaling changes the finite scores, and the mask removes the future columns from the selected row. The single head spans the full d_model. The 6 columns drawn for x, Q, K, V and Z are a schematic of that width; the course tensor has D = 512 columns. The widget shows the order of operations and how the shapes flow.

Layout inspired by S. Raschka.

Answer A1

Why does the scale matter?

Think it through in three steps.

- `Var(q·k)` as `d_k` grows?
- Effect on the softmax output?
- Effect on the gradients?

Scaling keeps the softmax out of saturation

dot-product variance

$$\mathrm{Var}(q \cdot k) = d_k$$

softmax Jacobian

$$\frac{\partial s_i}{\partial z_j} = s_i \left(\delta_{ij} - s_j\right)$$

- Assumes independent, zero-mean, unit-variance coordinates
- Large `d_k` : widely spread scores
- Near one-hot softmax: gradients near 0
- ÷√d_k restores unit variance (Vaswani et al., 2017)

▼ Why the scale is √d_k

Formula (3) divides every score by $\sqrt{d_k}$. The reason is an argument about the untrained model. Without the division, the scores spread more the wider the head is. A widely spread row makes the softmax nearly one-hot. A nearly one-hot softmax passes almost no gradient back to the scores. The argument concerns initialization only: training changes the weights, and afterwards nothing keeps the coordinates independent or the scores small.

Assumptions

Fix one query position and one key position, with input vectors $x, x' \in \mathbb{R}^D$. The randomness in the argument is the random initialization of the weights, with the inputs held fixed.

- **A1. Inputs of unit RMS.** Every input vector has a root mean square (RMS) of about 1:

$$\frac{1}{D}\sum_{i=1}^{D} x_i^2 \approx 1, \qquad \text{so} \qquad \sum_{i=1}^{D} x_i^2 \approx D.$$

  The block makes this approximately true: it normalizes the input of every branch to this scale, with a learned gain that starts at 1. The Transformer Block tab shows how.
- **A2. Independent random weights.** At initialization, the entries of W_Q and W_K are independent random variables with mean 0 and variance σ^2, and W_Q is drawn independently of W_K. The initialization schemes below all draw weights this way.
- **A3. A variance-preserving scale.** The weight variance is $\sigma^2 = 1/D$. Why this value is the natural choice follows from step 1.
- **A4. A row spreads like one score.** For one initialization, the scores of one row spread across its keys about as much as one score spreads across initializations. This holds approximately

when the positions of a row carry nearly independent inputs with RMS about 1. Correlated inputs, such as similar embeddings, spread less.

Two facts from probability are used below:

$$\operatorname{Var}\Big(\sum_{j=1}^{n} Y_j\Big) = \sum_{j=1}^{n} \operatorname{Var}(Y_j) \quad \text{for uncorrelated } Y_1, \ldots, Y_n, \tag{F1}$$

$$\operatorname{Var}(UW) = \operatorname{Var}(U)\operatorname{Var}(W) \quad \text{for independent } U, W \text{ with mean } 0. \tag{F2}$$

Step 1: one query coordinate

Claim. Under A1 and A2, every coordinate of the query has mean 0 and variance $\operatorname{Var}(q_j) \approx \sigma^2 D$, which is 1 under A3.

Coordinate j of the query is a weighted sum of D weights, $q_j = \sum_{i=1}^{D} x_i \, (W_Q)_{ij}$. Each weight has mean 0, so q_j has mean 0, and

$$\begin{aligned}
\operatorname{Var}(q_j) &= \operatorname{Var}\Big(\sum_{i=1}^{D} x_i \, (W_Q)_{ij}\Big) \\
&= \sum_{i=1}^{D} x_i^2 \operatorname{Var}\big((W_Q)_{ij}\big) && \text{F1 and A2: independent terms, } x \text{ fixed} \\
&= \sigma^2 \sum_{i=1}^{D} x_i^2 \\
&\approx \sigma^2 D && \text{A1} \\
&= 1 && \text{A3.}
\end{aligned}$$

The same computation gives $\operatorname{Var}(k_j) \approx 1$ for the key. A1 is the first approximation of the argument.

Variance-preserving initialization

A **variance-preserving initialization** chooses σ^2 so that a layer's output coordinates have the same variance as its inputs. The calculation above shows that this requires $\sigma^2 = 1/\text{fan_in}$, where the fan-in is the number of inputs each output coordinate sums over, here D. This is **LeCun initialization**, $\operatorname{Var}(W) = 1/\text{fan_in}$ (LeCun et al., 1998). Xavier (or Glorot) initialization sets $\operatorname{Var}(W) = 2/(\text{fan_in} + \text{fan_out})$, which also gives $1/D$ for a square $D \times D$ projection (Glorot & Bengio, 2010). He (or Kaiming) initialization sets $\operatorname{Var}(W) = 2/\text{fan_in}$, twice LeCun's value. The factor 2 makes up for a ReLU, which sets about half of the activations feeding the next layer to zero (He et al., 2015).

Step 2: the dot product

Claim. Under A1 to A3, $\mathrm{Var}(q \cdot k) \approx d_k$, and the mean of $q \cdot k$ is 0.

The dot product is a sum of d_k products, $q \cdot k = \sum_{j=1}^{d_k} q_j k_j$. For each j, the coordinates q_j and k_j are independent, because W_Q and W_K are, and both have mean 0. So each product has mean 0 and variance $\mathrm{Var}(q_j)\,\mathrm{Var}(k_j)$. Two different products are uncorrelated. For $j \neq j'$,

$$\mathbb{E}[q_j k_j \, q_{j'} k_{j'}] = \mathbb{E}[q_j q_{j'}]\,\mathbb{E}[k_j k_{j'}] = 0,$$

because the columns j and j' of W_Q are independent with mean 0, which makes $\mathbb{E}[q_j q_{j'}] = 0$. The variances therefore add:

$$\begin{aligned} \mathrm{Var}(q \cdot k) &= \sum_{j=1}^{d_k} \mathrm{Var}(q_j k_j) && \text{F1: products uncorrelated} \\ &= \sum_{j=1}^{d_k} \mathrm{Var}(q_j)\,\mathrm{Var}(k_j) && \text{F2: independent, mean } 0 \\ &\approx d_k && \text{step 1.} \end{aligned}$$

This is the first formula on the slide. It holds for a query and a key from the same position as well as from different positions. Vaswani et al. state it under the same assumption: the components of the query and the key are independent random variables with mean 0 and variance 1 (Vaswani et al., 2017). The mean stays 0. Only the spread grows with d_k.

Dividing every score by $\sqrt{d_k}$ divides its variance by d_k:

$$\mathrm{Var}\left(\frac{q \cdot k}{\sqrt{d_k}}\right) = \frac{\mathrm{Var}(q \cdot k)}{d_k} \approx 1.$$

With the scale, the initial scores are of order one whatever d_k is.

Step 3: from widely spread scores to a saturated row

Heuristic. Under A1 to A4, the unscaled scores of one row spread across its keys with a standard deviation of about $\sqrt{d_k}$, which is about 22.6 at the course width $d_k = 512$.

The variance of step 2 is taken over random initializations, for one fixed pair of inputs. The softmax sees something different: the spread across the keys of one row, for one initialization. A4 equates the two, and it is the second approximation of the argument.

The softmax depends only on the differences between scores, because adding the same constant to every score leaves its output unchanged. Two scores 10 apart already give a

probability ratio of $e^{10} \approx 22{,}000$, so a row whose scores spread by about 22.6 puts nearly all its weight on its largest score. An illustrative simulation, summarized in the table below, draws every score independently from a Gaussian with mean 0, for 400 rows of 1,024 keys (seed 0). Real scores only approximate this, since all scores of a row share one query. At a spread of 22.6, the largest weight in a row averages 0.90, and 71% of the rows put more than 0.9 on a single key.

Step 4: a saturated softmax passes almost no gradient

Claim. If a row is nearly one-hot, every entry of its softmax Jacobian is nearly zero, and almost no gradient reaches its scores.

Let $z \in \mathbb{R}^S$ be the scores of one row and $s = \mathrm{softmax}(z)$ its weights, $s_i = e^{z_i} / \sum_l e^{z_l}$. Differentiating with the quotient rule gives the **softmax Jacobian**, the second formula on the slide:

$$\frac{\partial s_i}{\partial z_j} = \frac{\delta_{ij}\, e^{z_i} \sum_l e^{z_l} - e^{z_i} e^{z_j}}{\left(\sum_l e^{z_l}\right)^2} = s_i \left(\delta_{ij} - s_j\right),$$

where δ_{ij} is 1 when $i = j$ and 0 otherwise. This is an exact identity.

Suppose the row is nearly one-hot on key m: $s_m \approx 1$ and $s_i \approx 0$ for every $i \neq m$. Then every entry of the Jacobian is nearly zero:

- the diagonal entry for m is $s_m(1 - s_m) \approx 0$;
- the other entries in row m are $-s_m s_j$, which contain a factor $s_j \approx 0$;
- every entry in a row $i \neq m$ contains the factor $s_i \approx 0$.

By the chain rule, the gradient of the loss $\mathcal{L}$ with respect to the scores is

$$\frac{\partial \mathcal{L}}{\partial z_j} = \sum_{i=1}^{S} \frac{\partial \mathcal{L}}{\partial s_i} \frac{\partial s_i}{\partial z_j},$$

so almost no gradient reaches the scores, and through them almost none reaches W_Q and W_K.

What the scale buys

The simulation measures how much gradient a row can pass with the Frobenius norm of its Jacobian, $\|J\|_F = \left(\sum_{i,j} J_{ij}^2\right)^{1/2}$:

Score standard deviation	Mean largest weight in a row	Rows with more than 0.9 on one key	Median Jacobian norm	Rows with Jacobian norm below 10^{-3}
1 (scaled scores)	0.016	0%	0.050	0%

Score standard deviation	Mean largest weight in a row	Rows with more than 0.9 on one key	Median Jacobian norm	Rows with Jacobian norm below 10^{-3}
7.5 (course initialization, unscaled)	0.70	28%	0.31	2%
22.6 (LeCun initialization, unscaled)	0.90	71%	0.021	33%

At a spread of 22.6, a third of the rows pass essentially no gradient, and the median norm falls to 0.021. Over seeds 0 to 4 that median ranges from about 0.01 to 0.03.

A median alone does not separate saturated rows from flat ones. Flat rows pass little gradient too: in a nearly uniform softmax over many keys every s_i is small, so at a spread of 1 the median norm is only 0.050, although no row is saturated. What the scale buys is an initial spread that does not depend on d_k: a wide head starts with the same spread as a narrow one, and no row starts saturated.

The course model starts from a smaller constant

The Week 2 model keeps PyTorch's default initialization for `nn.Linear`, which draws every weight uniformly from $U(-1/\sqrt{\text{fan_in}},\ 1/\sqrt{\text{fan_in}})$ (PyTorch Contributors, 2026). This replaces A3. A uniform distribution on $[-a, a]$ has variance $a^2/3$, and the computation of steps 1 and 2 then gives

$$\begin{aligned} \sigma^2 = \frac{a^2}{3} = \frac{1}{3\,\text{fan_in}} = \frac{1}{3D} \qquad & a = 1/\sqrt{\text{fan_in}}, \\ \text{Var}(q_j) \approx \sigma^2 D = \tfrac{1}{3} \qquad & \text{step 1}, \\ \text{Var}(q \cdot k) \approx d_k \cdot \tfrac{1}{3} \cdot \tfrac{1}{3} = \frac{d_k}{9} \qquad & \text{step 2}. \end{aligned}$$

That is a weight standard deviation of about 0.026 at $D = 512$, against 0.044 for LeCun initialization. At $d_k = 512$, the unscaled scores have a standard deviation of $\sqrt{512/9} \approx 7.5$, and the scaled scores about $1/3$. The middle row of the table shows the result: much sharper rows than with the scale, but only 2% of them with a Jacobian norm below 10^{-3}, and the median Jacobian norm of 0.31 is the largest of the three settings. At initialization, the course model's unscaled softmax would give sharp rows that still pass gradient.

GPT-2 and nanoGPT start from similar constants. GPT-2, and models built on its code such as nanoGPT, draw most weights from a normal distribution with a fixed standard deviation of 0.02 (OpenAI, 2019) (Karpathy, 2025). There are exceptions: GPT-2's position embedding uses 0.01, and nanoGPT scales its residual projections to $0.02/\sqrt{2\,n_{\text{layer}}}$. At $D = 512$, a standard deviation of 0.02 gives $\text{Var}(q \cdot k) \approx 21$, a spread of about 4.6. The course's 0.026 lies between that and LeCun's 0.044.

The initialization sets the constant, but in every case $\mathrm{Var}(q \cdot k)$ grows in proportion to d_k. The saturation argument is about that growth: at a fixed initialization a wider head spreads its scores further, and the scale removes the dependence on the width. Scores that grow during training are a separate problem, which Week 5 takes up together with initialization in depth.

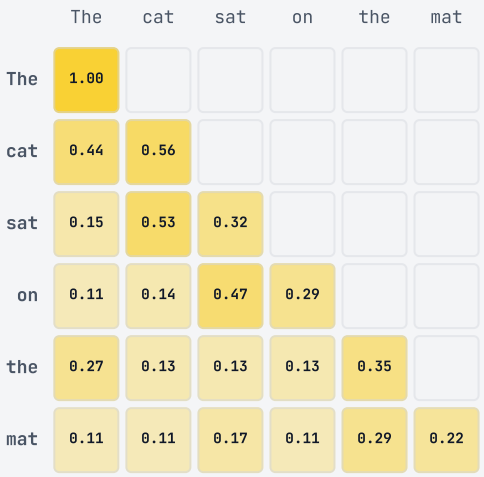
What Vaswani et al. claim

Vaswani et al. "suspect" that for large d_k the dot products grow large in magnitude and push the softmax into regions with extremely small gradients. They scale by $1/\sqrt{d_k}$ to counteract this (Vaswani et al., 2017). Their footnote gives step 2, starting from coordinates of unit variance. The saturation is their explanation of why the growth matters.

Masking, visualized

- Rows: queries; columns: keys
- Mask off: weight flows to the future
- Last row: unchanged, it sees everything

ILLUSTRATIVE Options: scaling ON (÷√d_k, d_k = 16) causal mask ON



Rows: queries; columns: keys.

The visible probabilities in each row sum to 1. Click a row to highlight it. Shaded cells are masked (j > i); the model cannot attend to future tokens. With the mask on, check that the columns j > i of the selected row are shaded and get no probability, while the diagonal stays available. Turn the mask off without changing the row, and the probability spreads over the future keys that are now reachable. Then toggle scaling to see its softer or sharper redistribution on its own. The sentence and scores are the flow diagram's synthetic fixture, so the probabilities are illustrative, not learned or measured.

▼ Compare masked and unmasked rows

Without a mask, a training position could read the tokens after it. Those tokens do not exist yet during autoregressive generation, where the model produces one token at a time from the tokens before it. The causal mask removes them during training as well, so the model learns under the same conditions it is used in.

The heatmap holds the attention weights of a short sentence, with one row per query and one column per key. With the causal mask on, each row keeps its weight on its own position and the positions before it. With the mask off, the weight of a row spreads onto the future keys it can now reach.

The last row does not change. The last query may already read every key, because no key comes after it, so the mask adds 0 to every score in that row. Its scores, and therefore its

softmax, are the same with the mask on or off. Every earlier row changes, because its softmax now shares the weight with keys that were masked before.

ILLUSTRATIVE Options: scaling ON (÷√d_k, d_k = 16) causal mask OFF

	The	cat	sat	on	the	mat
The	0.42	0.12	0.12	0.09	0.12	0.12
cat	0.25	0.32	0.12	0.09	0.12	0.12
sat	0.10	0.36	0.22	0.10	0.10	0.10
on	0.08	0.11	0.37	0.23	0.11	0.11
the	0.24	0.11	0.11	0.11	0.31	0.11
mat	0.11	0.11	0.17	0.11	0.29	0.22

Rows: queries; columns: keys.

The visible probabilities in each row sum to 1. Click a row to highlight it. With the mask on, check that the columns j > i of the selected row are shaded and get no probability, while the diagonal stays available. Turn the mask off without changing the row, and the probability spreads over the future keys that are now reachable. Then toggle scaling to see its softer or sharper redistribution on its own. The sentence and scores are the flow diagram's synthetic fixture, so the probabilities are illustrative, not learned or measured.

The causal mask turned off: every row except the last now puts weight on later keys. The last row is unchanged, because no key comes after it.

The causal mask hides future tokens

$$M_{ij} = \begin{cases} 0 & j \le i \\ -\infty & j > i \end{cases} \tag{4}$$

- Diagonal allowed: row `i` reads token `i`
- Row `i` predicts raw token `i+1`
- Add before softmax: future gets 0 (Vaswani et al., 2017)

▼ **Why the causal diagonal is allowed**

Formula (4) defines the Week 2 mask as an additive matrix. For query row i and key column j, $M_{ij} = 0$ when $j \le i$ and $M_{ij} = -\infty$ when $j > i$. Since $e^{-\infty} = 0$, adding the mask before the softmax gives every future position a probability of exactly zero (Vaswani et al., 2017).

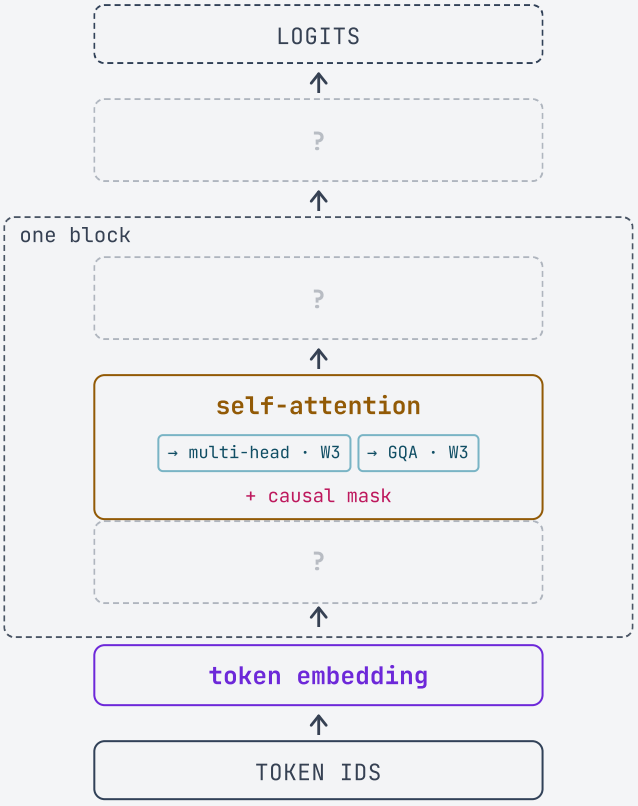
The diagonal stays allowed. Input row i holds raw token i, and the label paired with that row is raw token $i + 1$. Reading the current input token is not reading its target, so row i may use token i while it predicts token $i + 1$.

In code, build the upper-triangular mask of future positions, broadcast it over the `[B,S,S]` scores, add it after scaling, and apply the softmax last. This assumes every row has at least one allowed key, which the diagonal guarantees here. Week 3 covers the cases that need more: padding, boundaries between packed documents, boolean versus additive mask conventions, finite stand-ins for $-\infty$, fully blocked rows, and resets of positions and of the loss.

Attention returns to width D

- `W_O` : `[B,S,d_k]` to `[B,S,D]`

Attention mixes positions



▼ Why the output returns to width D

The attention branch mixes information from the allowed earlier positions and projects the result back to the model width. Its value blend `H` has shape `[B,S,d_k]`. Multiplying by $W_O \in \mathbb{R}^{d_k \times D}$ sums over the d_k axis and gives `O` of shape `[B,S,D]`. The batch and sequence axes are unchanged, and the last axis returns to the model width. In the Week 2 configuration $d_k = D$, but W_O is still a learned projection.

In the diagram, the arrow leaving the attention branch has the width of the main path. Matching `[B,S,D]` shapes are what make the residual addition in the Transformer Block tab possible. Week 3 covers several heads and how their outputs are concatenated.

Attention moves information across positions. The MLP, later in the block, transforms each position on its own.

Positional Embeddings

Without position, attention sees a set

- Permute inputs: outputs permute the same way
- The mask gives order a boundary (Haviv et al., 2022)
- Still no position vector per row

▼ Attention without a mask ignores order

Self-attention without a mask and without a position signal is **permutation-equivariant**: reordering the input rows reorders the output rows in the same way, and changes nothing else.

Claim. Let P be an $S \times S$ permutation matrix and X the input states of one sequence. Then $\mathrm{Attention}(PX) = P\ \mathrm{Attention}(X)$.

Permuting the inputs permutes the queries, keys and values, $Q' = PXW_Q = PQ$, and likewise $K' = PK$ and $V' = PV$. Write $A(\cdot)$ for the row-wise softmax. Permuting the rows and the columns of a score matrix Z in the same way permutes its softmax in the same way:

$$A(PZP^\top) = P\,A(Z)\,P^\top.$$

Then

$$\begin{aligned} A\Big(\frac{Q'K'^\top}{\sqrt{d_k}}\Big)V' &= A\Big(P\,\frac{QK^\top}{\sqrt{d_k}}\,P^\top\Big)PV \\ &= P\,A\Big(\frac{QK^\top}{\sqrt{d_k}}\Big)P^\top PV \qquad \text{softmax under permutation} \\ &= P\,A\Big(\frac{QK^\top}{\sqrt{d_k}}\Big)V \qquad P^\top P = I \\ &= P\ \mathrm{Attention}(X). \end{aligned}$$

The equations therefore contain no notion of position or order. Each output row is a weighted sum over all the keys, and a sum does not depend on the order of its terms.

What the mask adds

A causal mask breaks the symmetry, because row i can see only the keys $j \le i$, so a causal decoder is not a bag of tokens. Within one attention layer, the last query still sums over all earlier tokens, so permuting them leaves its output unchanged. A decoder can infer some position information from how much each row can see, and Haviv et al. show that causal

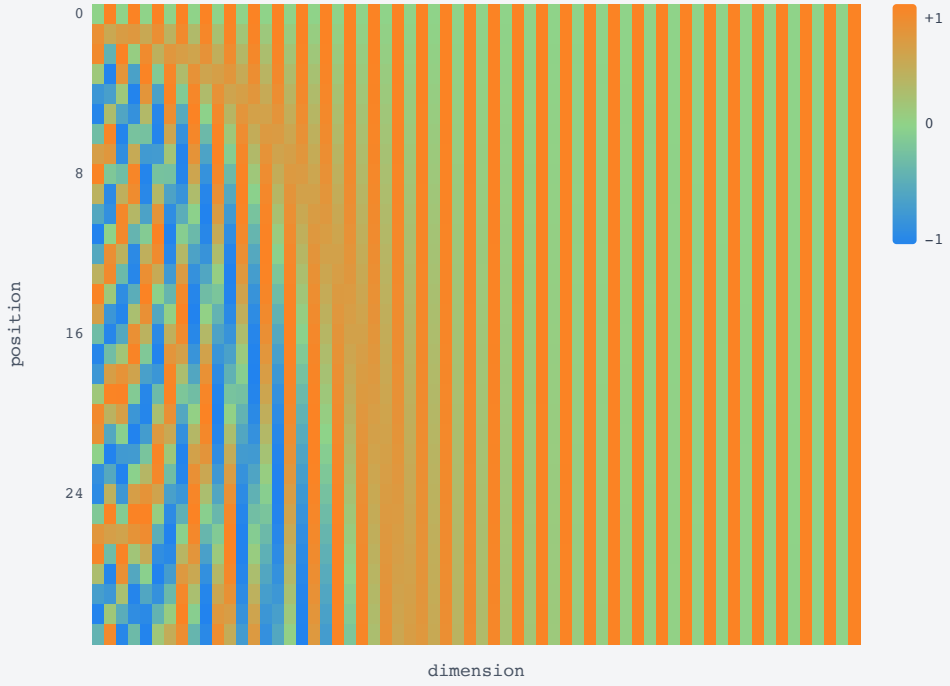
language models without any positional encoding (NoPE) still learn positional information (Haviv et al., 2022). The mask still gives no row a direct position vector. The Week 2 model supplies one explicitly, with the fixed sinusoidal signal on the next slide, which also keeps the implementation easy to trace. Week 3 returns to NoPE.

Sinusoids add position without parameters

$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/D}}\right), \quad PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{2i/D}}\right) \tag{5}$$

- Fixed: zero learned parameters (Vaswani et al., 2017)
- Fast and slow frequency pairs
- Offset `k`: a rotation of each pair

ILLUSTRATIVE d_model: 32 64 128



Each row is one of the 32 displayed positions, and each adjacent pair of columns is one sine/cosine frequency pair. The legend maps values from -1 through 0 to +1. The fast pairs on the left change quickly down the position axis; the slow pairs on the right change gradually. No two positions share a pattern. The widths 32, 64 and 128 only size this illustration; the course model uses D = 512.

▼ A fixed offset is a rotation

Formula (5) adds a fixed signal to every position. It has no learned parameters: the table is computed from the position, the pair index i and D, and training never changes it (Vaswani et al., 2017). Each adjacent pair of dimensions holds one sine and one cosine at a single frequency, $\omega_i = 10000^{-2i/D}$. The first pairs have frequencies near 1 and change quickly from one position to the next. The last pairs have frequencies near $1/10000$ and change slowly.

Claim. Moving from position pos to position $pos + k$ rotates every pair by an angle that depends on k and i, but not on pos.

Write $\theta = pos \cdot \omega_i$ for the angle of pair i at position pos, so the pair is $(\sin\theta, \cos\theta)$. An offset of k positions adds the angle $\alpha = k\,\omega_i$. The angle-addition identities give

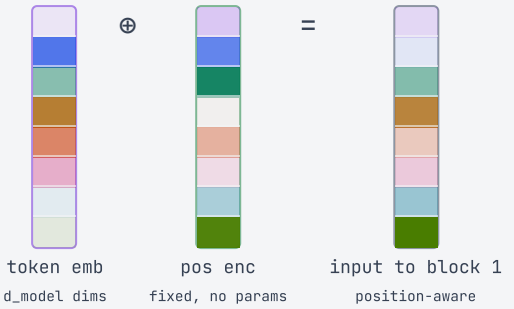
$$\begin{pmatrix} \sin(\theta+\alpha) \\ \cos(\theta+\alpha) \end{pmatrix} = \begin{pmatrix} \sin\theta\cos\alpha + \cos\theta\sin\alpha \\ \cos\theta\cos\alpha - \sin\theta\sin\alpha \end{pmatrix} = \begin{pmatrix} \cos\alpha & \sin\alpha \\ -\sin\alpha & \cos\alpha \end{pmatrix} \begin{pmatrix} \sin\theta \\ \cos\theta \end{pmatrix}.$$

The matrix maps the pair at angle θ to the pair at angle $\theta + \alpha$, a turn by α on the unit circle. It contains k and ω_i but not pos, so the same offset acts as the same rotation at every position, independently within each pair. Here the rotation acts on the added signal. RoPE, in Week 3, rotates the queries and keys instead.

Adding position keeps the width Read on your own

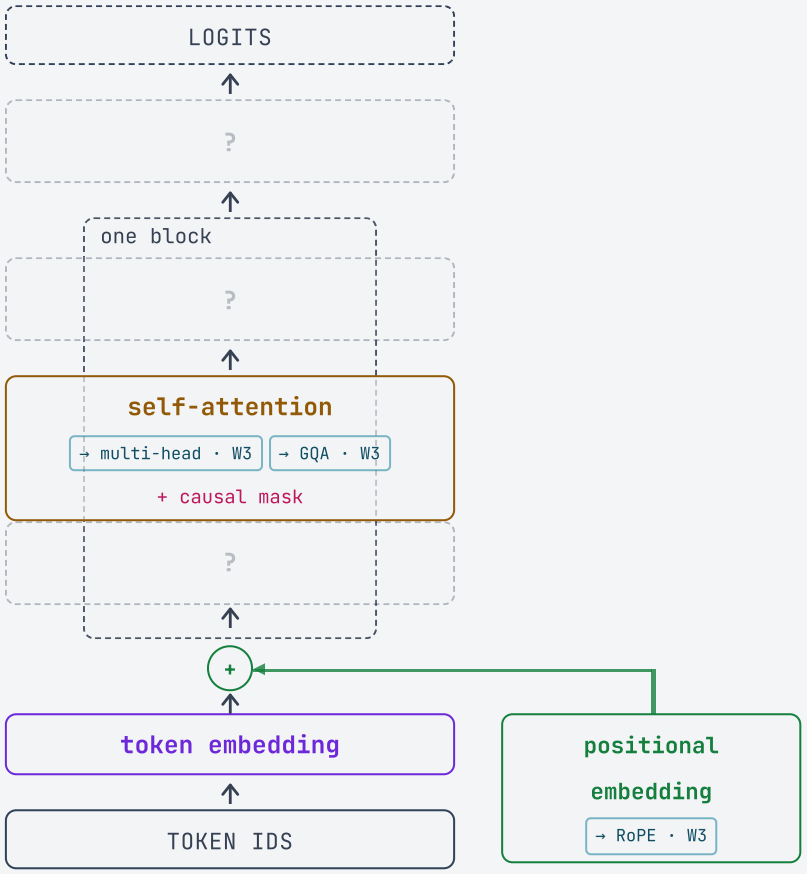
- `[B,S,D] + [S,D]`: same shape
- RoPE rotates Q and K: Week 3

ILLUSTRATIVE positional encoding is added once at the input (same d_model, element-wise)



token embedding (learnable) positional encoding (fixed) position-aware input

Sinusoidal positional encoding is added once, element by element, to the token embedding at the input, so the width stays d_model. It is fixed, has no parameters and encodes absolute position. RoPE, in Week 3, brings in relative position instead, at every layer and inside the QK product. Each column shows 8 dimensions with deterministic synthetic values. The widget clamps the displayed sums to [-1, 1] so that opacity stays readable. The clamp is for display only: the model adds token and position vectors without clipping.



▼ Addition preserves the model width

Let the token tensor X have shape `[B,S,D]` and let the positional table PE have shape `[S,D]`, broadcast over the batch. Elementwise addition gives

$$Y_{bsd} = X_{bsd} + \mathrm{PE}_{sd}$$

for every batch index b, position s and dimension d. No axis is concatenated or reduced, so Y keeps the shape `[B,S,D]`. The diagram sends the result into the same D-wide block path.

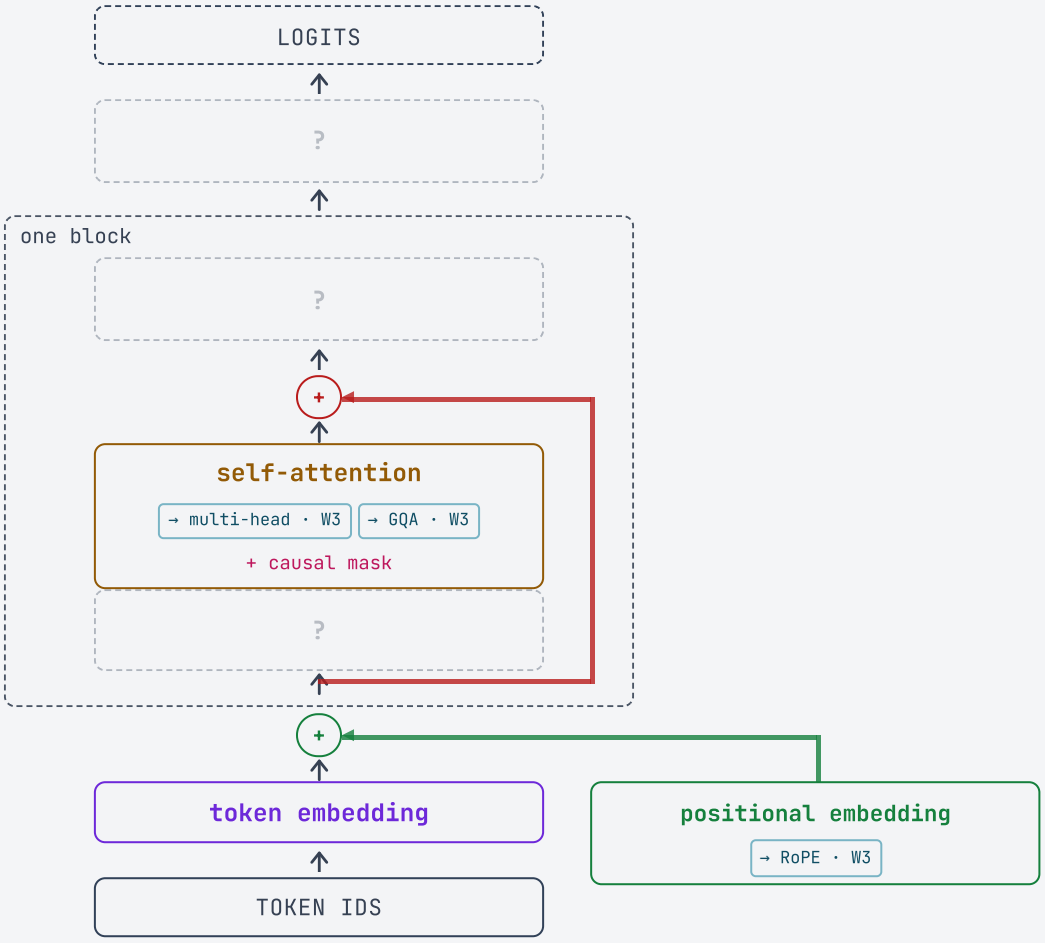
RoPE appeared as a preprint in 2021 and as a journal publication in 2024 (Su et al., 2024). Week 3 derives and implements its rotation of the queries and keys.

Transformer Block

Residuals keep the input and add an update

$$x \leftarrow x + \mathrm{Sublayer}(\mathrm{RMSNorm}(x)) \tag{6}$$

- Branch learns an update: `x + F(x)`
- **Identity path** carries gradients
- Eases optimization, no guarantee (He et al., 2016)



▼ A residual branch learns an update to its input

A block should change its state without forcing every sublayer to reconstruct the input. A direct additive path achieves this: the branch learns an update, and the previous state stays available. That is the residual update on the slide, $x + F(x)$. In formula (6), the sublayer is attention or the MLP, and RMSNorm (next slide) rescales its input.

Write one residual update as $y = x + F(x)$, with x and $F(x)$ in the same space $\mathbb{R}^D$, and let J_F be the Jacobian of F. Differentiating gives

$$\frac{\partial y}{\partial x} = I + J_F.$$

The identity matrix is a direct contribution to the Jacobian, so information and gradients do not have to pass through the branch alone. That helps explain why the bypass eases optimization, though $I + J_F$ can still amplify, shrink or cancel particular directions (He et al., 2016).

In the diagram, the state splits into two routes. One passes through self-attention, the other goes around it, and they rejoin at the plus node. The sum is the next state, at the same width. Elhage et al. call this running sum the **residual stream**: every component reads from it and writes to it by adding its output (Elhage et al., 2021).

RMSNorm rescales each branch input

$$\text{RMSNorm}(x) = g \odot \frac{x}{\sqrt{D^{-1}\sum_{i=1}^{D} x_i^2 + \epsilon}}, \qquad \epsilon = 10^{-5} \tag{7}$$

- **Root mean square normalization** (RMSNorm)
- No mean-centering; learned gain
- Pre-norm: better gradients at init (Xiong et al., 2020)
- The course still uses warmup

show: LayerNorm **RMSNorm**

$$y = \gamma \odot \frac{x}{r}$$
$$r = \sqrt{\text{mean}(x^2) + \epsilon}$$

RMSNorm drops: ~~mean centering (μ)~~ ~~bias (β)~~

RMSNorm keeps the learned rescaling and drops the recentering and the bias. Whether RMSNorm runs faster depends on the implementation. Zhang & Sennrich 2019 (RMSNorm).

▼ RMSNorm controls the scale each branch sees

Every residual addition can change the scale of the state that later branches read. We therefore normalize the input of each branch and leave the residual path itself untouched.

For a token vector $x \in \mathbb{R}^D$, RMSNorm reduces over the width axis. It computes the root mean square

$$r(x) = \sqrt{\frac{1}{D}\sum_{i=1}^{D} x_i^2 + \epsilon},$$

divides every coordinate by the same $r(x)$, and then multiplies coordinate by coordinate with the learned gain $g \in \mathbb{R}^D$, as in formula (7). It does not subtract the mean. The course sets $\epsilon = 10^{-5}$, a small constant that keeps the division safe when x is close to zero.

An example: x = (3, 4)

Set ϵ to 0 and the gain to its initial value $g = (1, 1)$. For $x = (3, 4)$,

$$r(x) = \sqrt{\frac{3^2 + 4^2}{2}} = \sqrt{12.5} \approx 3.536,$$
$$\frac{x}{r(x)} \approx (0.849,\ 1.131).$$

The result has a root mean square of 1, the scale that assumption A1 of the √d_k derivation relies on.

RMSNorm and LayerNorm

LayerNorm, the older choice, subtracts the mean, divides by the standard deviation, and adds a learned bias after the gain. RMSNorm drops the mean-centering and the bias and keeps the gain. Whether it runs faster depends on the implementation (Zhang & Sennrich, 2019). In the comparison widget, switch between LayerNorm and RMSNorm and compare the denominators, then the mean-centering and bias terms that RMSNorm crosses out.

Where the norm goes

Placing the norm before the branch is called pre-norm. In the Pre-LN settings that Xiong et al. analyze, gradients at initialization are better behaved, and in their experiments Pre-LN Transformers trained without warmup matched Post-LN baselines trained with it (Xiong et al., 2020). That is a result for their settings, and the course uses learning-rate warmup anyway. Where the norm goes is an architecture choice: OLMo 2 normalizes the output of each branch instead of its input (OLMo Team, 2025).

show: LayerNorm RMSNorm

$$y = \gamma \odot \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta$$
$$\mu = \mathrm{mean}(x), \qquad \sigma^2 = \mathrm{var}(x)$$

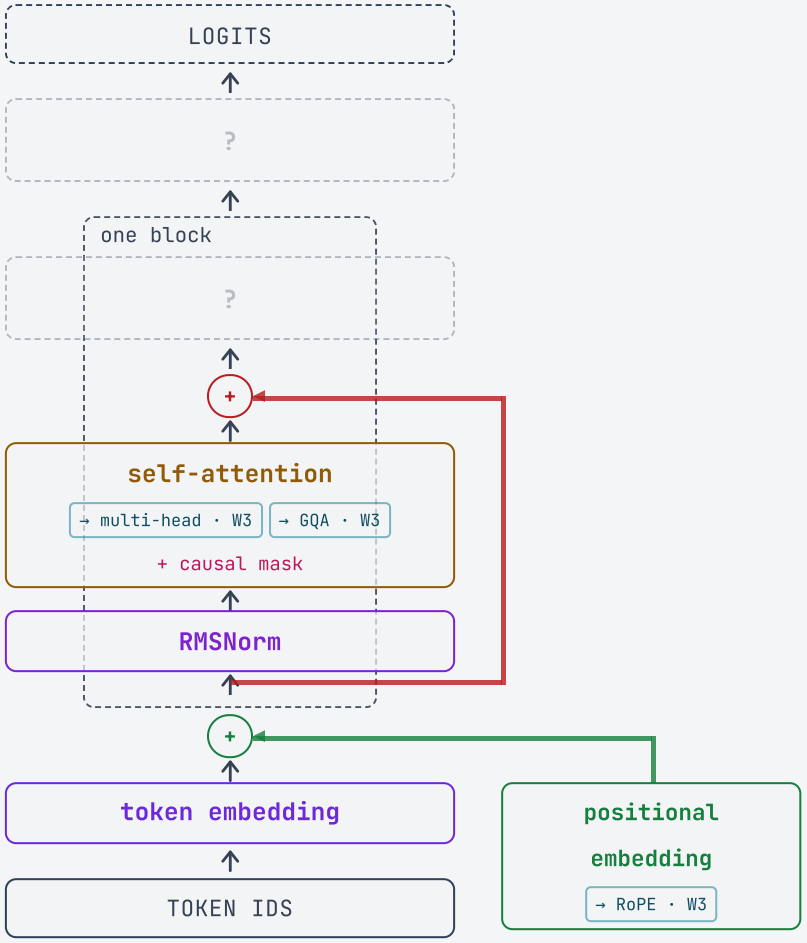
LayerNorm includes: mean-centering (μ) bias (β)

RMSNorm keeps the learned rescaling and drops the recentering and the bias. Whether RMSNorm runs faster depends on the implementation. Zhang & Sennrich 2019 (RMSNorm).

LayerNorm for comparison: it subtracts the mean μ, divides by the standard deviation and adds a bias β. RMSNorm divides by the root mean square and drops μ and β.

RMSNorm acts on the branch input only

- Norm before attention and MLP
- Residual route stays direct



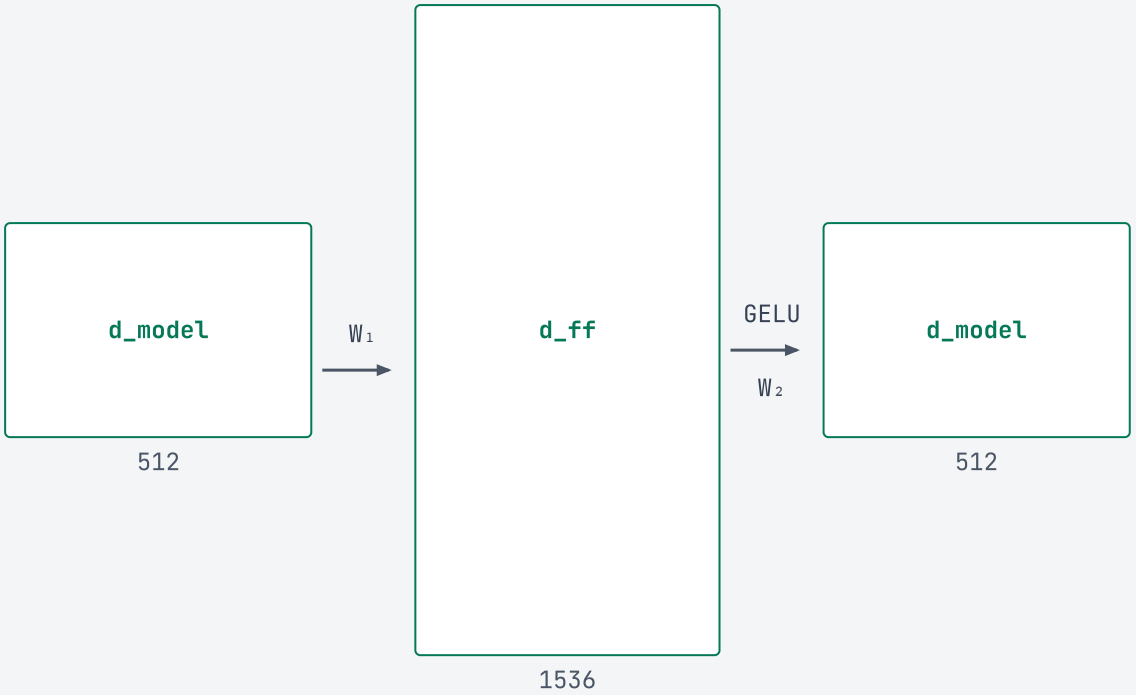
▼ One norm per branch, none on the residual route

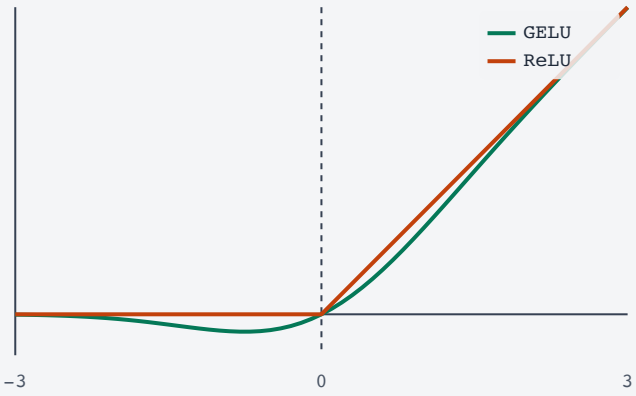
In the diagram, RMSNorm is on the input to the attention branch. The residual route bypasses it and rejoins at the addition. The MLP branch is built the same way, so the block has a norm before each of its two branches and a direct route around each.

Attention mixes; the MLP transforms

(8) $$\mathrm{MLP}(x) = \mathrm{GELU}(xW_1)W_2, \qquad D \to 3D \to D$$

- **MLP**: a nonlinearity per token
- `D → 3D → D`, GELU, no bias (Hendrycks & Gimpel, 2016)
- Same weights at every position





$$\mathrm{GELU}(x) = x\,\Phi(x)$$

$$\Phi(x) = \tfrac{1}{2}\left[1 + \mathrm{erf}\left(x/\sqrt{2}\right)\right]$$

$$\mathrm{GELU}(x) \approx 0.5\,x\left(1 + \tanh\left[\sqrt{2/\pi}\,\left(x + 0.044715\,x^3\right)\right]\right)$$

Phi is the standard normal CDF. The curve uses the tanh approximation.

The MLP expands to d_ff = 3 * d_model, applies GELU and projects back. The original Transformer used ReLU and a 4x expansion. The course baseline uses GELU, a 3x expansion, and bias-free matrices. Hendrycks & Gimpel 2016.

▼ Why the block needs an MLP

Claim. For fixed attention weights a_{ij}, attention is linear in its inputs. The output of position i is

$$o_i = \sum_j a_{ij}\, v_j W_O = \sum_j a_{ij}\, x_j W_V W_O.$$

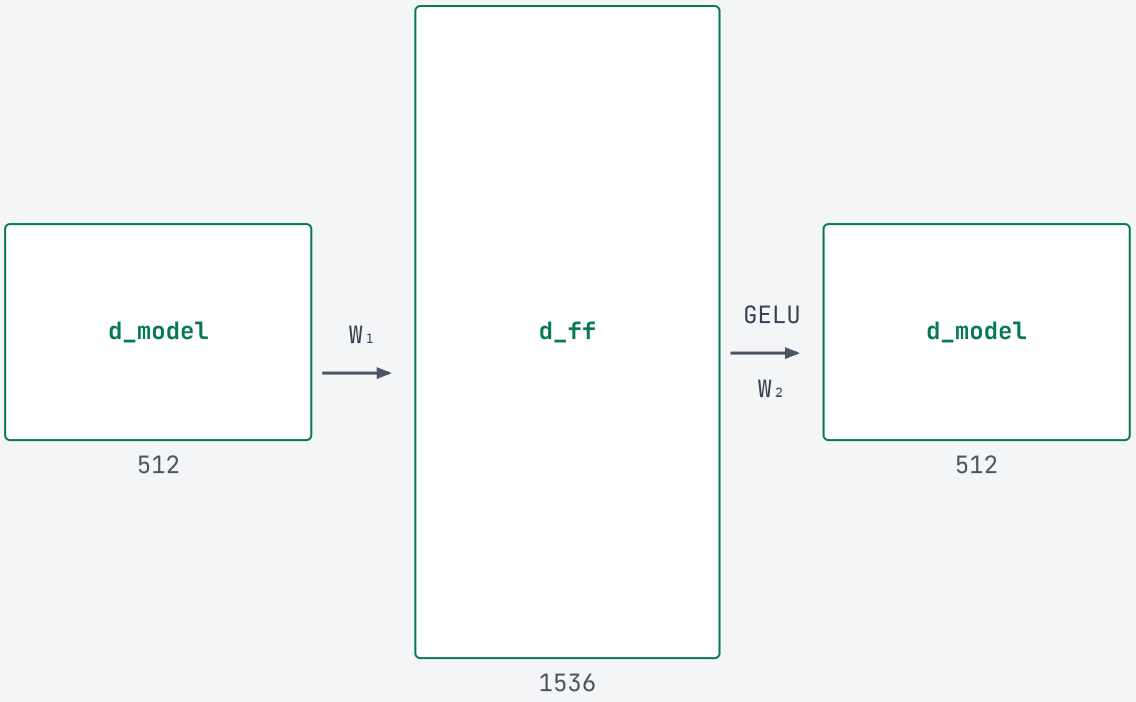
The weights themselves depend on the inputs, through the softmax of the scores. Even so, each output is a weighted average of the projected inputs $x_j W_V W_O$: attention chooses how to mix the positions, but it builds no new nonlinear feature from any one of them. The block needs a nonlinear transform as well, and the MLP supplies it, applied to each position on its own.

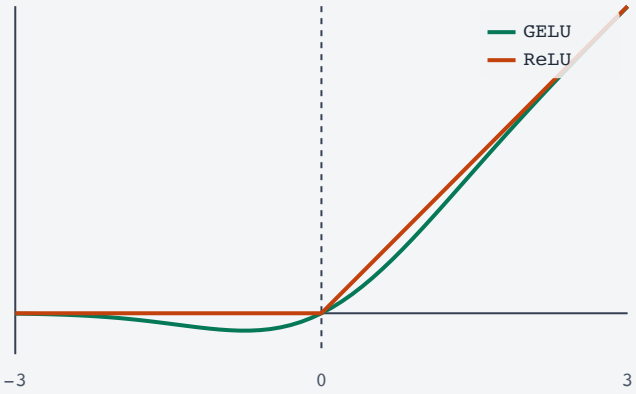
The course MLP has two matrices and no biases. Starting from hidden states X of shape `[B,S,D]`, the matrix $W_1 \in \mathbb{R}^{D\times 3D}$ expands each position to `[B,S,3D]`. GELU keeps that shape, and $W_2 \in \mathbb{R}^{3D\times D}$ projects back to `[B,S,D]` for the residual addition. The same two matrices are used at every batch and sequence index, so the MLP transforms the features within a position and never mixes positions. The course sets the hidden width d_{ff} to $3D$, which gives the widths $512 \to 1536 \to 512$.

GELU is $x\,\Phi(x)$, where Φ is the cumulative distribution function of the standard normal distribution. The often-quoted tanh expression is an approximation to it (Hendrycks & Gimpel, 2016). The original Transformer used ReLU and a wider hidden layer, $d_{ff} = 4D$ (Vaswani et al., 2017). Many recent models replace this MLP with a gated one, SwiGLU, among them Llama 3 (Grattafiori et al., 2024). The reading slide "Gated MLPs multiply two projections" describes it. The exercise uses the GELU MLP.

Attention or the MLP?

Which holds more of the block's parameters?





$$\mathrm{GELU}(x) = x\,\Phi(x)$$

$$\Phi(x) = \tfrac{1}{2}\left[1 + \mathrm{erf}\left(x/\sqrt{2}\right)\right]$$

$$\mathrm{GELU}(x) \approx 0.5\,x\left(1 + \tanh\left[\sqrt{2/\pi}\,\left(x + 0.044715\,x^3\right)\right]\right)$$

Phi is the standard normal CDF. The curve uses the tanh approximation.

The MLP expands to d_ff = 3 * d_model, applies GELU and projects back. The original Transformer used ReLU and a 4x expansion. The course baseline uses GELU, a 3x expansion, and bias-free matrices. Hendrycks & Gimpel 2016.

Answer A2

Gated MLPs multiply two projections Read on your own

SwiGLU MLP

$$\mathrm{SwiGLU}(x) = \big(\mathrm{Swish}(xW_{gate}) \odot xW_{up}\big)W_{down}$$

- A gate multiplies each hidden unit
- Same cost: hidden width × 2/3
- Lower loss, no explanation offered (Shazeer, 2020)
- Later analysis: gates speed up optimization (Lyu et al., 2026)

▼ SwiGLU: a gated MLP at the same cost

The course MLP applies GELU to one projection of the input. A **gated** MLP computes two projections and multiplies them coordinate by coordinate, so that one acts as a gate on the other. SwiGLU, a gated variant that many recent models use (Week 3 lists them), has three matrices: $W_{gate}, W_{up} \in \mathbb{R}^{D\times d_{ff}}$ map to the hidden width and $W_{down} \in \mathbb{R}^{d_{ff}\times D}$ maps back (Shazeer, 2020):

$$\mathrm{SwiGLU}(x) = \big(\mathrm{Swish}(xW_{gate}) \odot xW_{up}\big)W_{down},$$

where $\odot$ is the elementwise product, $\mathrm{Swish}(u) = u\,\sigma(u)$, and σ is the logistic sigmoid. Each hidden unit is the product of two learned linear features of the input, one of them passed through Swish. Where the gate is near zero, the unit is switched off for this token, whatever the other projection says.

The same cost with a smaller hidden width

Three matrices hold $3D\,d_{ff}$ parameters instead of $2D\,d_{ff}$. To compare the two MLPs at the same cost, Shazeer shrinks the hidden width by a factor of 2/3 (Shazeer, 2020):

$$\begin{aligned} 3D\,d_{ff}^{gated} &= 2D\,d_{ff} && \text{equal parameter counts} \\ d_{ff}^{gated} &= \tfrac{2}{3}\,d_{ff} && \text{divide by } 3D. \end{aligned}$$

The forward FLOPs scale the same way, because each of the three matrices costs $2BS\,D\,d_{ff}^{gated}$ FLOPs. For the course model, the GELU MLP with $d_{ff} = 3D$ would become a SwiGLU MLP with $d_{ff} = 2D = 1024$, still $6D^2$ parameters. Shazeer's baseline had $d_{ff} = 3072$ at a model width of 768, a $4D$ MLP, so his gated layers used $d_{ff} = 2048$, which is $8D/3$. Released models pick their own widths: Llama 3 uses SwiGLU with $3.5D$ at 8B and 70B and $3.25D$ at 405B (Grattafiori et al., 2024).

Why use the gate

Shazeer compared the MLP variants inside T5, an encoder-decoder Transformer pretrained to fill in deleted spans of text, with every model matched for parameters and computation. After 524,288 training steps, the held-out log-perplexity was 1.677 for the ReLU baseline, 1.679 for GELU and 1.636 for SwiGLU. Even the bilinear variant, $(xW_{gate} \odot xW_{up})W_{down}$ with no activation function at all, reached 1.648, which suggests that much of the gain comes from the product itself. After fine-tuning, the gated variants scored best on most of the GLUE and SuperGLUE language-understanding tasks, results that Shazeer describes as noisy (Shazeer, 2020).

The paper gives no mechanism. Its conclusion reads: "We offer no explanation as to why these architectures seem to work; we attribute their success, as all else, to divine benevolence" (Shazeer, 2020). Shazeer's case is empirical: a lower loss for the same parameters and compute.

Later work offers a partial explanation. Lyu et al. analyze two-layer networks in the neural tangent kernel regime, in which a very wide network trains like a fixed kernel method. They show that the gate makes this kernel better conditioned, so that gradient descent converges faster (Lyu et al., 2026). In their ViT and GPT-2 experiments the gated variants changed the gap between training and held-out loss little, and the authors conclude that the main benefit of gating is faster optimization. An older argument points the same way. In the gated linear unit $(xW) \odot \sigma(xV)$ that Dauphin et al. introduced, the gradient reaches the linear branch xW scaled only by the gate value $\sigma(xV)$, without the shrinking derivative of a second activation, and their gated convolutional language models converged faster to a better perplexity with it than with a tanh-gated unit (Dauphin et al., 2017). Both results come from settings much simpler than a large Transformer trained with Adam, so they make faster optimization a plausible explanation for large language models, which neither paper tests directly.

Week 3 lists models that use SwiGLU. The course keeps the two-matrix GELU MLP, and the exercise uses it. Gating also appears in attention: in Week 4's reading slide "An output gate lets a head write almost nothing", a sigmoid gate multiplies each head's attention output.

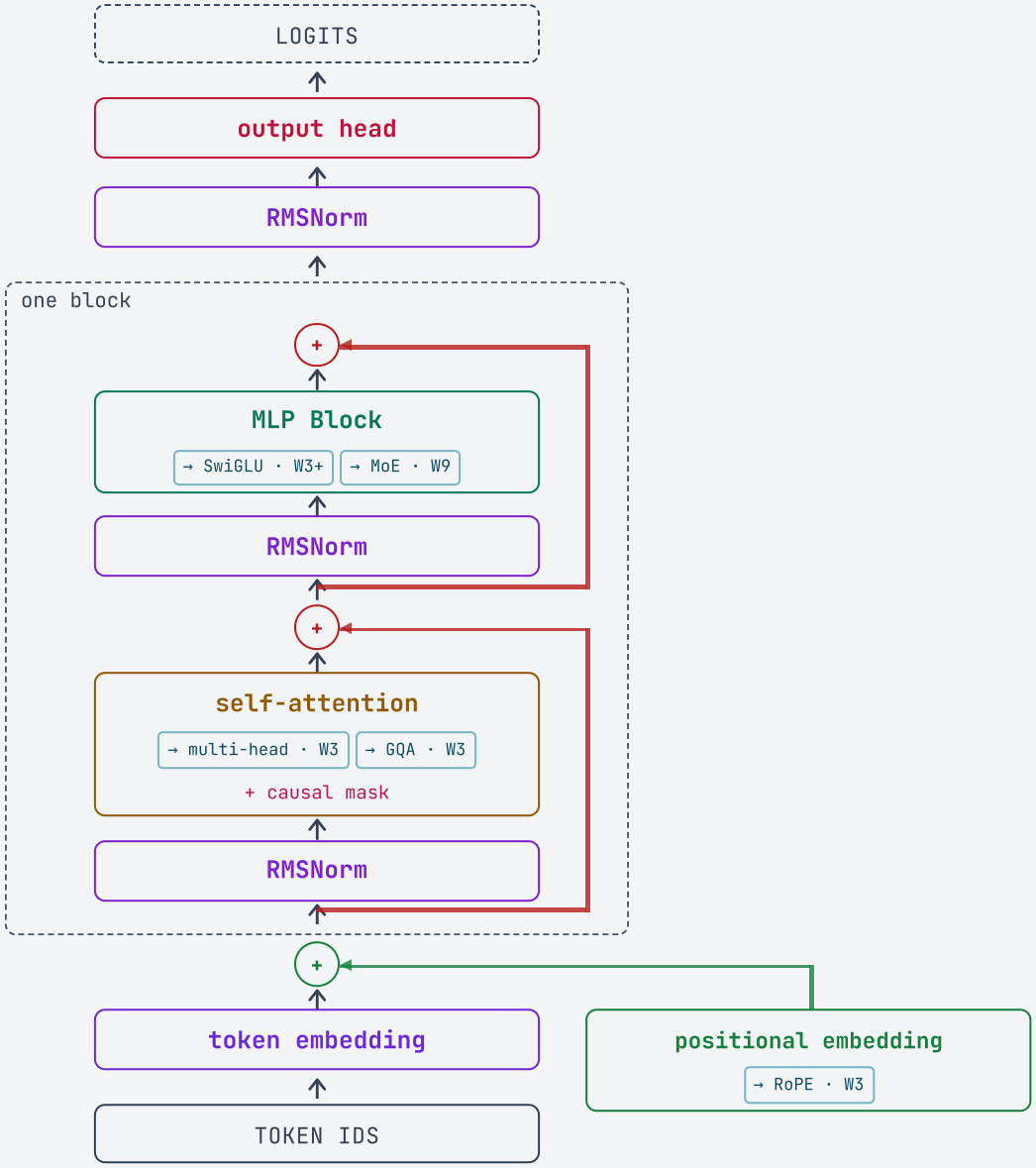
Minimal GPT

26

Four lines assemble the model

```
x = token_embedding(inputs) + sinusoidal_position
x = x + attention(rmsnorm_1(x), causal_mask)
x = x + mlp(rmsnorm_2(x))
logits = tied_head(final_rmsnorm(x))
```

- `[B,S] → [B,S,D] → [B,S,V]`



▼ The model as a chain of shape-preserving updates

The four lines are the whole model, and the diagram shows the same path in data-flow order. The tensor starts as token IDs of shape `[B,S]` and ends as logits of shape `[B,S,V]`, with

hidden states of shape `[B,S,D]` in between.

The token lookup and the addition of the sinusoidal positions create `x` with shape `[B,S,D]`. The first RMSNorm feeds self-attention, the causal mask limits it, and its result is added back to `x`. The second RMSNorm feeds the GELU MLP, whose width-D result is added back again. Each of the two lines inside the block updates the same `[B,S,D]` state.

After the block, the final RMSNorm still returns `[B,S,D]`. The tied head uses the transposed embedding matrix as a projection from width D to V vocabulary scores and produces the logits. Read the diagram from the token IDs upward through the two residual updates, then the final normalization and the output head. This model has one block. Week 3 stacks several.

Tying shares one table for input and output

(9) $$P_{tied} = VD + L(4D^2 + 2Dd_{ff}) + (2L + 1)D$$

Configuration	Parameters
tied, one block	`6,817,280`
untied: add `VD=4,194,304`	`11,011,584`

- **One table**: lookup in, scores out (Press & Wolf, 2017)
- Saves `VD = 4,194,304`
- A course choice, not a rule

▼ Where the parameters of the one-block model are

The input table and the output head both connect the vocabulary to the width D. The table maps token IDs to vectors, and the head scores every token from a vector. With **weight tying**, one matrix $E \in \mathbb{R}^{V \times D}$ does both jobs: its rows are the lookup table, and its transpose produces the output scores. That saves VD parameters. Press and Wolf recommend tying for language models and report lower perplexity with it (Press & Wolf, 2017). Tying is a course choice. The configurations of Qwen3-8B, DeepSeek-V3 and OLMo 3 7B are untied (Qwen3, DeepSeek-V3, OLMo 3).

The count

Formula (9) counts the trainable arrays of a tied model with L blocks:

- the tied embedding and output head share VD parameters;
- each block's attention has the four projections W_Q, W_K, W_V, W_O. With Week 2's one wide head, $d_k = D$, they hold $4D^2$, and for a general head width $4D\,d_k$;
- each block's bias-free MLP holds $2D\,d_{ff}$;
- each block has two RMSNorm gains and the model one final gain, together $(2L + 1)D$;
- the fixed sinusoidal positions add nothing.

For the course configuration, $V = 8192$, $D = 512$, $d_{ff} = 1536$ and $L = 1$:

$$\begin{aligned} P_{tied} &= \underbrace{8192 \cdot 512}_{\text{embedding and head}} + \underbrace{4 \cdot 512^2}_{\text{attention}} + \underbrace{2 \cdot 512 \cdot 1536}_{\text{MLP}} + \underbrace{3 \cdot 512}_{\text{gains}} \\ &= 4{,}194{,}304 + 1{,}048{,}576 + 1{,}572{,}864 + 1{,}536 \\ &= 6{,}817{,}280. \end{aligned}$$

Untying adds a separate output head of another $VD = 4{,}194{,}304$ parameters, for $11{,}011{,}584$ in total. All these totals assume one head with $d_k = D$, bias-free matrices, a $D \to 3D \to D$ MLP, two gains per block plus a final one, and fixed sinusoidal positions.

One block exaggerates the vocabulary

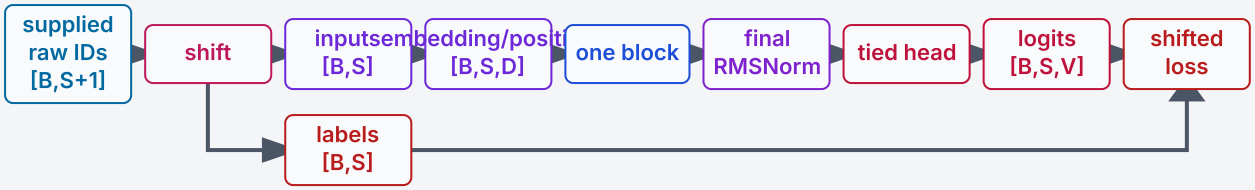
In the one-block model, the shared table holds 62% of all parameters, 4,194,304 of 6,817,280. That share is a consequence of having a single block, because the table does not grow with depth. At eight blocks, the depth of the Week 3 exercise, the same count gives

$$4{,}194{,}304 + 8\,(1{,}048{,}576 + 1{,}572{,}864) + 17 \cdot 512 = 25{,}174{,}528$$

parameters, of which the table is 17%. Every further block lowers the share again.

One window, from shift to loss

- Raw `[B,S+1]` → shift → inputs, labels
- Logit row `i` scored against token `i+1`



▼ Follow one token window from shift to loss

A raw window of token IDs with shape `[B,S+1]` is shifted once into inputs and labels of shape `[B,S]`. The inputs pass through the hidden states `[B,S,D]` to the logits `[B,S,V]`, and the loss compares the logits with the labels.

Take one batch row with the raw token IDs 10, 20, 30 and 40. Its raw length is $S + 1 = 4$, so the shift gives $S = 3$ positions:

Position i	0	1	2
Input	10	20	30
Label	20	30	40

The embedding and the position signal turn the inputs into hidden states of shape `[1,3,D]`. The block and the final RMSNorm keep that shape, and the tied head produces logits of shape `[1,3,V]`. Logit row 0 is scored against token 20, row 1 against 30 and row 2 against 40: row i predicts raw token $i + 1$.

The horizontal diagram shows the same path. From the box of raw IDs, the shift splits into two branches. The upper branch carries the inputs through embedding and position, the block, the final RMSNorm, the tied head and the logits. The lower branch carries the labels straight to the loss.

Training repeats five steps

$$\mathcal{L} = -\frac{1}{BS}\sum_{b=1}^{B}\sum_{i=0}^{S-1}\log p_\theta(x_{b,i+1} \mid x_{b,\le i}) \tag{10}$$

1. Zero gradients
2. Forward to logits
3. Cross-entropy against shifted labels
4. Backward
5. Optimizer step and log

▼ Why the loss indices line up

Formula (10) is the Week 1 objective, computed by the model you just assembled. One training step runs five operations in order: zero the gradients, run the forward pass to the logits, compute the cross-entropy against the shifted labels, run the backward pass, then take the optimizer step and log. Training repeats this step.

The indices line up as follows. Let a raw row be $x_{b,0}, \ldots, x_{b,S}$. Input position i receives $x_{b,i}$, and its label is $x_{b,i+1}$. The forward pass returns exactly S logit rows, so the cross-entropy pairs `logits[b, i, :]` with `labels[b, i]` for every i from 0 to $S-1$. That is the meaning of the conditional probability in formula (10): the state built from the prefix up to raw position i is scored against the next raw token.

The factor $1/(BS)$ assumes that all BS target positions count. If padding excludes some targets, let $\mathcal{T}$ be the set of retained pairs (b, i) and average over those only:

$$\mathcal{L} = -\frac{1}{|\mathcal{T}|}\sum_{(b,i)\in\mathcal{T}}\log p_\theta(x_{b,i+1} \mid x_{b,\le i}).$$

The same padding mask chooses the targets and counts them for the denominator.

Trade-offs Framework

30

Five axes, five kinds of evidence

Question	Evidence
Quality	shifted cross-entropy loss
Compute	analytic matmul FLOPs; supported-op profiler estimate
Memory	analytic buckets; peak allocated and reserved bytes
Time	synchronized wall-clock tokens/s
Money	priced runtime outside the required exercise

▼ **Every decision trades five things**

Every engineering decision trades quality, time, compute, memory and money. In this lecture, the shifted cross-entropy loss is the quality signal. Analytic counts of FLOPs and bytes predict the cost, and profiler events, wall-clock throughput and allocator peaks test those predictions.

The table pairs each axis with its evidence. Compute has two kinds: the matmul FLOPs you count by hand, and the profiler's estimate for the operators it supports. Memory has your analytic buckets and the peak allocated and reserved bytes that the allocator reports. Time is synchronized wall-clock throughput in tokens per second. Money is runtime multiplied by a price, which the required exercise does not ask for. Each estimate has its own measurement, and the tutorial walks through the measurements.

Two configurations with the same FLOP count can still run at different speeds. Week 4 explains why.

Counting forward FLOPs

Component	Week 2 forward matmul FLOPs
Q, K, V, output projections	$8BSLD^2$
score and weighted-value matmuls	$4BS^2LD$
two MLP matmuls	$4BSLDd_{ff}$
tied vocabulary projection	$2BSVD$

- Multiply-add: 2 **FLOPs**
- `L` : number of blocks (here 1)
- Per token: head 8.4M, block 7.3M

▼ Derive each forward FLOP term

FLOP counts let you compare architectures before running them, and they show where the compute goes as a model grows. The symbols are B for the batch size, S for the sequence length, L for the number of blocks, D for the model width, V for the vocabulary size and d_{ff} for the MLP width. Today's model has one block, $L = 1$.

The counting rule

Multiplying an $m \times k$ matrix by a $k \times n$ matrix computes mn dot products of length k, which is mnk multiply-adds. Counting a multiply-add as two FLOPs, the product costs

$$2mkn \text{ FLOPs.}$$

The four terms

Apply the rule to all BS token rows in all L blocks:

$$\begin{aligned}
\text{Q, K, V and output projections:} &\quad 4 \cdot 2\,(BS)\,D\,D \cdot L = 8BSLD^2, \\
\text{scores } QK^\top \text{ and weighted values:} &\quad 2 \cdot 2\,B\,S\,S\,D \cdot L = 4BS^2LD, \\
\text{two MLP matrices:} &\quad 2 \cdot 2\,(BS)\,D\,d_{ff} \cdot L = 4BSLDd_{ff}, \\
\text{tied output head:} &\quad 2\,(BS)\,D\,V = 2BSVD.
\end{aligned}$$

The projection term uses Week 2's single wide head, $d_k = D$. For a general head width it is $8BSLDd_k$. The score term has S^2 because each of the S queries meets each of the S keys. Dense causal attention still computes the full score and weighted-value products unless a backend changes the computation, so a triangular mask alone does not halve this count.

The numbers

At $B = 32$, $S = 1024$, $L = 1$, $D = d_k = 512$, $d_{ff} = 1536$ and $V = 8192$:

Term	FLOPs per batch forward	FLOPs per token
projections	68,719,476,736	2,097,152
scores and weighted values	68,719,476,736	2,097,152
MLP	103,079,215,104	3,145,728
output head	274,877,906,944	8,388,608
total	515,396,075,520	15,728,640

The per-token column divides by the $BS = 32{,}768$ tokens of the batch. The block's three terms add up to 7,340,032 FLOPs per token, so in this model the output head costs more than the whole block: 8,388,608 against 7,340,032.

That comparison, too, is a consequence of the single block. The head's cost does not depend on L, while the block terms grow with it. At eight blocks, the depth of the Week 3 exercise, the blocks cost $8 \cdot 7{,}340{,}032 = 58{,}720{,}256$ FLOPs per token, and the head's 8,388,608 are 12.5% of the forward pass instead of 53%.

When attention costs as much as the MLP

A crossover needs a statement of what is compared with what. Setting only the quadratic score and weighted-value terms equal to the MLP gives

$$\begin{aligned} 4BS^2LD &= 4BSLDd_{ff} \\ S &= d_{ff} = 1536 \quad \text{divide by } 4BSLD. \end{aligned}$$

Setting all attention matmuls equal to the MLP gives

$$\begin{aligned} 8BSLD^2 + 4BS^2LD &= 4BSLDd_{ff} \\ 2D + S &= d_{ff} \qquad\qquad \text{divide by } 4BSLD \\ S &= d_{ff} - 2D = 512, \end{aligned}$$

which is $S = D$ for $d_{ff} = 3D$. At $S = 1024$, the two ratios to the MLP are $S/d_{ff} = 2/3$ and $(2D + S)/d_{ff} = 4/3$.

6NT is a coarse training estimate

(11) $$C \approx 6NT \ (\text{forward} \approx 2NT,\ \text{backward} \approx 4NT)$$

- Forward ≈ `2NT`, backward ≈ `4NT`
- `T`: training tokens; `D` stays width
- Omits attention's `S²`, norms, systems (Hoffmann et al., 2022)

▼ Why 6NT is only a first estimate

Let N be the number of model parameters and T the total number of training tokens. The letter D keeps its meaning as the model width.

Approximation. For a dense model whose cost is dominated by its parameters, training costs about

$$C \approx \underbrace{2NT}_{\text{forward}} + \underbrace{4NT}_{\text{backward}} = 6NT \ \text{FLOPs}.$$

The forward term counts every parameter in one multiply-add per token, which is 2 FLOPs. The backward pass computes gradients with respect to both the inputs and the weights, so it costs about twice the forward pass. Hoffmann et al.'s supplementary calculation makes the same backward-to-forward assumption and compares its component count with the common $6NT$ (Hoffmann et al., 2022).

Use $6NT$ to size a run. It does not replace the component count, because it compresses or leaves out the attention term that grows with S^2, the softmax and normalization work, recomputation, optimizer work, communication and implementation effects (Hoffmann et al., 2022).

The tied one-block model

With $N = 6{,}817{,}280$, the proxy gives $2N = 13{,}634{,}560$ forward FLOPs per token and $6N = 40{,}903{,}680$ training FLOPs per token. One batch step holds $T = BS = 32{,}768$ tokens, so

$$6N \cdot BS = 1{,}340{,}331{,}786{,}240 \ \text{FLOPs per step}.$$

The component count of the previous slide gives 15,728,640 forward FLOPs per token. Counting backward as twice that gives 47,185,920 training FLOPs per token, and 1,546,188,226,560 per step. The forward proxy falls 2,094,080 FLOPs per token, or 13.31%, short of the component count.

The gap has two exact sources. In the tied model, $2N$ counts every weight matrix at 2 FLOPs per parameter, which matches the projection, MLP and head terms exactly. What $2N$ misses is the score term, $4SLD$ per token, which has no parameters. It also counts $2(2L+1)D$ FLOPs for the norm gains, which do no matmul work:

$$15{,}728{,}640 - 13{,}634{,}560 = \underbrace{4 \cdot 1024 \cdot 1 \cdot 512}_{2{,}097{,}152} - \underbrace{2 \cdot 3 \cdot 512}_{3{,}072} = 2{,}094{,}080.$$

The score term grows with depth, while the head term, which $2N$ matches exactly, does not. At the eight blocks of the Week 3 exercise the proxy therefore falls 25% short.

Training memory has seven buckets

- Parameters and gradients: `P` each
- Adam: two FP32 moments; master optional
- Activations: batch, length, width, depth
- Attention matrices: `B·L·S²`
- Logits: `B·S·V`

▼ What each memory bucket scales with

Training memory goes to four kinds of tensors: parameters, gradients, optimizer state and saved activations. Let P be the parameter count and b the number of bytes per stored element, with a subscript for each kind of tensor.

- **Parameters,** the persistent model weights, take $P\,b_{param}$ bytes.
- **Gradients,** one per trained parameter, take $P\,b_{grad}$ bytes. Both buckets scale with the model size.
- **Adam's two moments** are kept in FP32, so they take $2P \cdot 4$ bytes.
- **An FP32 master copy** of the weights takes $P \cdot 4$ bytes, but only when the implementation keeps one.
- **Saved activations** grow with the batch size, the sequence length, the width and the depth. We estimate the part that is linear in the width as $\kappa\,BSDL\,b_{act}$ bytes, where κ is a coarse coefficient that the course fixes at 4. The number of tensors a real block saves for the backward pass depends on the implementation and is usually larger.
- **The attention matrices** take $n_{attn}\,BLS^2\,b_{attn}$ bytes, where n_{attn} counts the full `[B,S,S]` matrices kept at once: 2 for a conservative estimate of a naive implementation, 1 for a naive implementation in which you have checked that only one is kept, 0 for a fused backend.
- **The vocabulary logits,** when materialized, take $BSV\,b_{logits}$ bytes.

The last two activations get buckets of their own because they can dominate particular configurations. That makes seven buckets: parameters, gradients, two Adam moments, the optional master copy, width-linear activations, attention matrices and logits.

Different buckets need different remedies. Smaller batches reduce the activations. So does activation checkpointing, which recomputes saved activations during the backward pass instead of storing them. Sharding, in a later week, distributes the persistent state across devices.

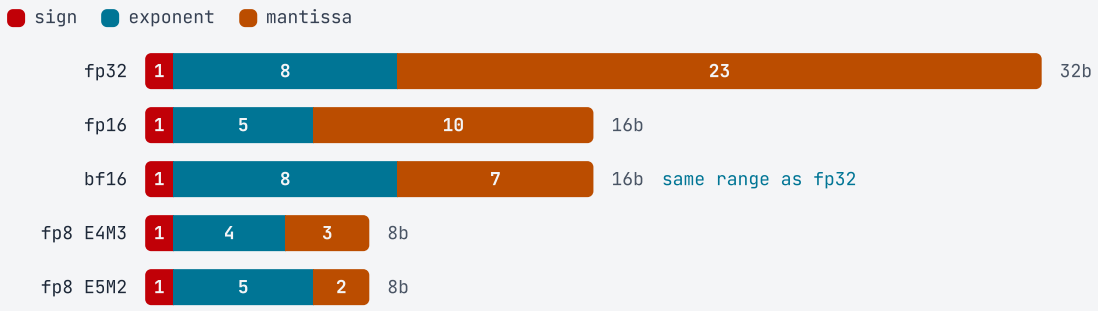
Which memory bucket is largest?

At B = 32, S = 1024 and BF16

Answer A3

Data type sets bytes, range and precision Read on your own

- Bytes: FP32 4, BF16/FP16 2, FP8 1
- BF16: FP32's range, less precision (Kalamkar et al., 2019)
- Storage dtype ≠ compute policy



Exponent bits set the dynamic range, and mantissa bits set the precision. bf16 keeps fp32's 8 exponent bits but has fewer fraction bits. How training uses these formats, FP8 scaling and stability included, is a Week 5 topic.

▼ The storage dtype leaves the compute choices open

FP32 uses 4 bytes per value, BF16 and FP16 use 2, and FP8 formats use 1. Lower precision reduces storage and can unlock faster hardware paths, but it also reduces the range or the precision. Read each format as three facts. The bytes per value follow from the bit count. The range is set largely by the exponent bits, and the precision within that range by the fraction bits. BF16 keeps the 8-bit exponent of FP32 with fewer fraction bits, so it has the range of FP32 with less precision (Kalamkar et al., 2019).

The bit cards say nothing about how a training kernel computes. The dtype a tensor is stored in does not fix the dtype of the accumulator, the kernel or the scaling recipe. Those are implementation and hardware choices, and you record them separately.

NVIDIA Transformer Engine documents one block-scaled FP8 path, in which every block of values has its own FP32 scale factor, by default a power of two (NVIDIA, 2026). FP8 appears here for comparison only. Week 5 covers runnable precision recipes.

Each estimate has its own measurement Read on your own

Question	Measurement
Which operators ran?	`torch.profiler` events (PyTorch Contributors, 2026)
Supported-op FLOPs?	`with_flops=True` estimate (PyTorch Contributors, 2026)
Tokens per second?	warmed, synchronized wall clock
Allocator peak?	`max_memory_allocated()`, `max_memory_reserved()` (PyTorch Contributors, 2026)

▼ A reproducible measurement sequence

Each estimate from this tab has its own measurement. `torch.profiler` records which operators ran, with their events and timing (PyTorch Contributors, 2026). Tokens per second come from a warmed, synchronized wall clock over several iterations. Allocator peaks come from `max_memory_allocated()` and `max_memory_reserved()`, queried after `reset_peak_memory_stats()` for a chosen scope (PyTorch Contributors, 2026). The tutorial walks through the three tools.

Fix the setup first

Before measuring, freeze and record the model, the batch, S, the dtype and compute policy, the backend and materialization choices, the data slice, the software and hardware versions, and the exact region you measure. Warm up that region before you collect evidence. Keep the analytic calculation, the profiler capture, the wall-clock timing, the allocator peaks and the memory snapshot as separate records, because they answer different questions.

Three measurements

For supported-operation FLOPs, profile a short, specified region with `torch.profiler.profile(..., with_flops=True)`. Report the covered operator events and the profiler's formula-based estimate. The profiler estimates FLOPs for the operators it supports from their shapes. It does not read a hardware FLOP counter (PyTorch Contributors, 2026).

For throughput, synchronize CUDA, time several iterations that are not profiled, and synchronize again. Divide the number of processed tokens, $n_{\text{iter}} \cdot B \cdot S$ for n_{iter} iterations of full batches, by the elapsed seconds, and report tokens per second together with the number of repetitions.

For allocator peaks, finish the warmup, synchronize, call `reset_peak_memory_stats()`, run the forward, backward and optimizer region you specified, synchronize, then query `max_memory_allocated()` and `max_memory_reserved()` (PyTorch Contributors, 2026). Allocated bytes are the live tensor allocations that PyTorch sees. Reserved bytes are the pool that the caching allocator manages. A memory snapshot shows the history and liveness of allocator-managed memory, but it can miss direct CUDA allocations (PyTorch Contributors, 2026). Compare only matching scopes and units, and use the materialization choices and the omitted work to investigate a gap. Week 2 does not use these numbers to place a step on a roofline or to compute model FLOPs utilization (MFU). Week 4 introduces both.

In the exercise, you calculate parameters, FLOPs and memory, compare the analytic and profiler estimates, measure throughput and peaks, sweep the supported FP32 and BF16 settings, and explain one FLOP gap and one memory gap.

Summary

37

How a model predicts the next token, and what it costs

1. `[B,S+1]` → `[B,S]` → logits `[B,S,V]`
2. Q, K, V `[B,S,d_k]`; scores `[B,S,S]`
3. ÷√d_k: unit variance at initialization
4. Mask: row `i` reads ≤ `i`, predicts `i+1`
5. `x + F(RMSNorm(x))`: attention, then MLP
6. Estimate, then measure: 6.8M, 872 MiB

▼ The answer to today's question

The question for today was how a model turns the context into a next-token prediction, and what that costs. A training example starts as a raw window of token IDs with shape `[B,S+1]`. One shift gives inputs and labels of shape `[B,S]`. The token embeddings and the sinusoidal position signal turn the inputs into states of width D, with shape `[B,S,D]`, and the tied head turns the final states into logits of shape `[B,S,V]`, whose cross-entropy against the labels trains the whole path.

Inside the block, attention mixes information across positions. For a head of width d_k, the queries, keys and values $Q = XW_Q$, $K = XW_K$ and $V = XW_V$ have shape `[B,S,d_k]`, and the scores and attention probabilities have shape `[B,S,S]`. The weighted values return to `[B,S,d_k]`, and W_O maps them back to `[B,S,D]`. If the query and key coordinates are independent with mean 0 and variance 1, dividing the scores by $\sqrt{d_k}$ keeps their variance of order one at initialization. The Week 2 model has one wide head, $d_k = D$. The causal mask allows the keys $j \leq i$, so row i reads the positions up to i and predicts position $i + 1$. The sinusoidal position values are added to the token embeddings.

A pre-norm block computes $x + F(\mathrm{RMSNorm}(x))$ twice: first with attention as F, then with the position-wise GELU MLP. A final RMSNorm and the tied output head turn the last state into logits.

The cost of this model is a set of numbers you can derive before running anything. It has 6,817,280 parameters. A forward pass costs about 15.7M FLOPs per token (15,728,640), and the analytic training memory at the calculator default is 872 MiB. With a single block, the vocabulary dominates all three costs: the shared table holds most of the parameters, the output head needs most of the forward FLOPs, and the logits take most of the memory. At the eight blocks of Week 3 it no longer does. These numbers are analytic: exact parameter counts, component forward FLOPs, the coarse $C \approx 6NT$ rule and the memory buckets. Profiler events and supported-op FLOPs, synchronized wall-clock tokens per second, and reset-scoped allocator peaks are measurements, each with its own scope. The exercise puts the two side by side and checks every estimate against its measurement.

Week 3 extends this single-head block into multi-head attention, grouped-query attention, and rotary position embeddings. Attempt the reflection questions without hints and bring your reasoning to the tutorial. Question 6 asks for an optional, ungraded prediction about those three changes. A tentative hypothesis is enough, and none of their mechanics are expected in Week 2.

38

To-dos until next week

Muddiest point

Do it right now! (2 min)

Before the tutorial

- Write down one question to discuss
- Think about the reflection questions

Self-study

- Read the reading slides and notes
- Explore the cited papers

Deadlines

- First exercise: Wednesday 28 October, 22:00
- Quiz 1: Friday 30 October

Week 3 extends this single-head block into multi-head attention, grouped-query attention, and rotary position embeddings.

Reflection questions

1. Trace the tensor shapes and shifted labels through one training example. Where can an off-by-one bug hide?
2. What information comes from token embeddings, positional encoding, and the causal mask respectively?
3. Why do pre-normalization and residual paths make a deep transformer easier to optimize?
4. When can analytic FLOP estimates, profiler counts, and measured wall time disagree?
5. What does tying the input embedding and output head save, and what assumption does it make?
6. Which parts of this single-head block must change for multi-head attention, grouped-query attention, and rotary position embeddings next week?

References

40

References: Attention, position and block architecture

- Dauphin, Y. N., Fan, A., Auli, M., & Grangier, D. (2017). Language Modeling with Gated Convolutional Networks. ICML 2017, PMLR 70, 933-941.
- Elhage, N., Nanda, N., Olsson, C., et al. (2021). A Mathematical Framework for Transformer Circuits. Transformer Circuits Thread.
- Glorot, X., & Bengio, Y. (2010). Understanding the difficulty of training deep feedforward neural networks. AISTATS 2010, PMLR 9, 249-256.
- Grattafiori, A., Dubey, A., Jauhri, A., et al. (2024). The Llama 3 Herd of Models. arXiv preprint arXiv:2407.21783.
- Haviv, A., Ram, O., Press, O., Izsak, P., & Levy, O. (2022). Transformer Language Models without Positional Encodings Still Learn Positional Information. Findings of the Association for Computational Linguistics: EMNLP 2022, 1382-1390.
- He, K., Zhang, X., Ren, S., & Sun, J. (2015). Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification. ICCV 2015, 1026-1034.
- He, K., Zhang, X., Ren, S., & Sun, J. (2016). Identity Mappings in Deep Residual Networks. ECCV 2016, 630-645.
- Hendrycks, D., & Gimpel, K. (2016). Gaussian Error Linear Units (GELUs). arXiv:1606.08415.
- LeCun, Y., Bottou, L., Orr, G. B., & Müller, K.-R. (1998). Efficient BackProp. In Neural Networks: Tricks of the Trade, LNCS 1524, 9-50. Springer.
- Lyu, X., Xu, Q., Yang, Z., Wen, P., & Huang, Q. (2026). The Devil is in the Condition Numbers: Why is GLU Better than non-GLU Structure?. ICML 2026, PMLR 306, 84030-84054.
- Press, O., & Wolf, L. (2017). Using the Output Embedding to Improve Language Models. EACL 2017.
- PyTorch Contributors. (2026). torch.nn.Linear, PyTorch 2.13. PyTorch API documentation.
- Shazeer, N. (2020). GLU Variants Improve Transformer. arXiv:2002.05202.
- Su, J., Ahmed, M., Lu, Y., Pan, S., Bo, W., & Liu, Y. (2024). RoFormer: Enhanced Transformer with Rotary Position Embedding. Neurocomputing, 568, 127063.
- Vaswani, A., Shazeer, N., Parmar, N., et al. (2017). Attention Is All You Need. NeurIPS 2017.

References: Normalization, model choices and cost

- DeepSeek-AI. (2025 revision for the 2024 model release). DeepSeek-V3 config.json. Official model configuration.
- Hoffmann, J., Borgeaud, S., Mensch, A., et al. (2022). An empirical analysis of compute-optimal large language model training. Advances in Neural Information Processing Systems 35 (NeurIPS 2022).
- Kalamkar, D., Mudigere, D., Mellempudi, N., et al. (2019). A Study of BFLOAT16 for Deep Learning Training. arXiv:1905.12322.
- Karpathy, A. (2025). nanoGPT: model.py. GitHub repository.
- OLMo Team. (2025). 2 OLMo 2 Furious. arXiv:2501.00656.
- Allen Institute for AI. (2025). OLMo-3-7B-Instruct config.json. Official model configuration.
- OpenAI. (2024 revision of the 2019 release). gpt-2: src/model.py. GitHub repository.
- Qwen Team. (2025). Qwen3-8B config.json. Official model configuration.
- Xiong, R., Yang, Y., He, D., et al. (2020). On Layer Normalization in the Transformer Architecture. ICML 2020.
- Zhang, B., & Sennrich, R. (2019). Root Mean Square Layer Normalization. NeurIPS 2019.

References: Measurement documentation

- NVIDIA. (2026). Using FP8 with block scaling. Transformer Engine documentation.
- PyTorch Contributors. (2026). CUDA semantics: Memory management. PyTorch documentation.
- PyTorch Contributors. (2026). Understanding CUDA Memory Usage. PyTorch documentation.
- PyTorch Contributors. (2026). PyTorch Profiler. PyTorch documentation.
- PyTorch Contributors. (2026). torch.profiler.profile. PyTorch API documentation.

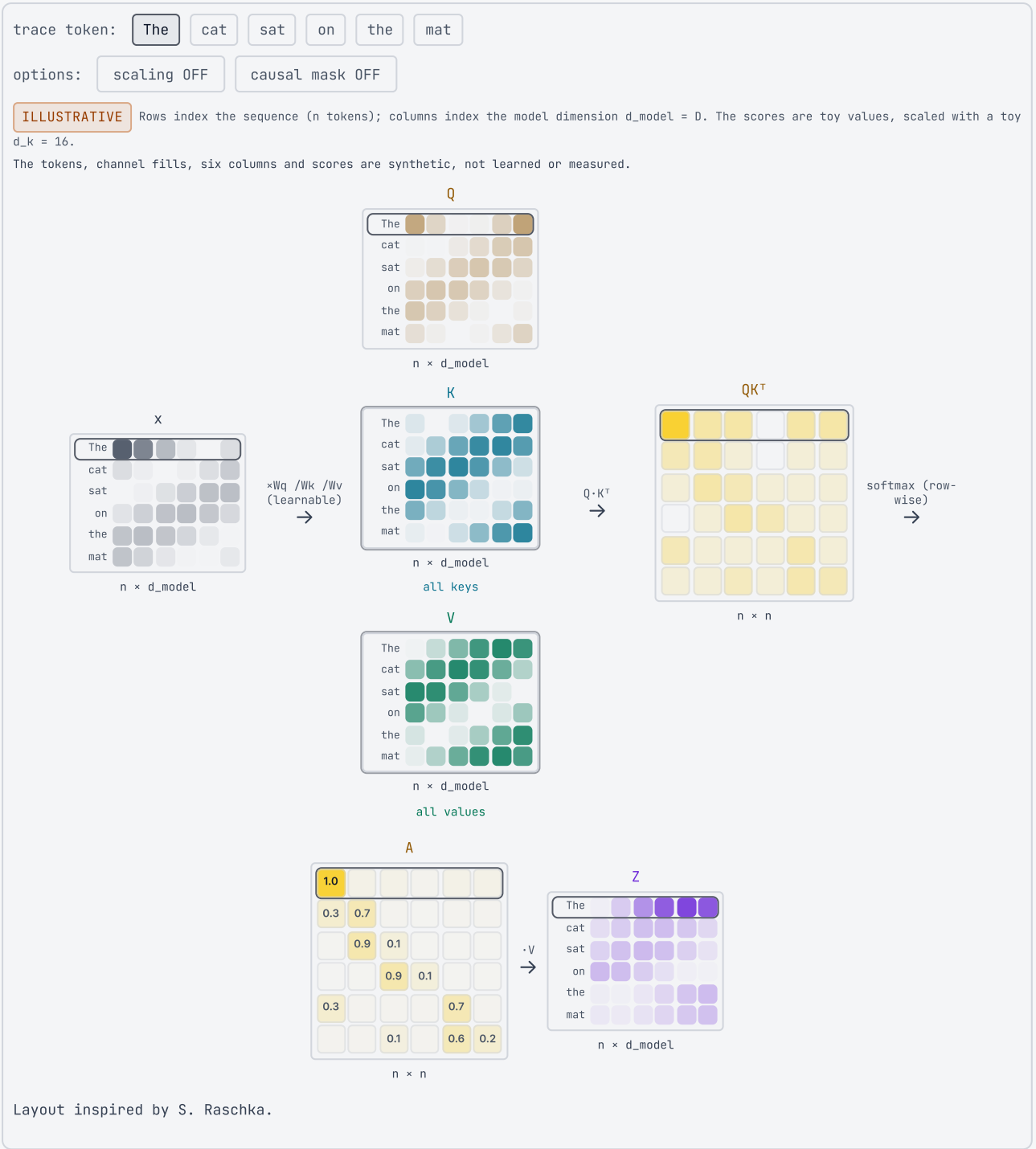
Image credits

- Ramelli, A. (1588). Crank-driven fountain pump with its shaft. *Le diverse et artificiose machine*, Figure LVI. Science History Institute. Public domain.
- Ramelli, A. (1588). Book wheel. *Le diverse et artificiose machine*, Figure CLXXXVIII. Wikimedia Commons. Public domain.
- Hevelius, J. (1673). Aerial telescope on a mast. *Machinae coelestis*, Fig. AA. Houghton Library, Harvard University. Public domain.
- Anonymous (1618). A giant balance in a council hall. *Op de Waag-Schaal*. Rijksmuseum, Amsterdam. CC0.
- Agricola, G. (1556). Washing gold-bearing gravel with a sluice and pan. *De re metallica*. National Library of Poland. Public domain.

Answers

A1 · Without scaling, attention rows sharpen

- Sharper: one key takes most weight
- Unscaled scores are 4× larger here



Scaling off: every score is four times larger, so each row puts more of its weight on its highest-scoring key.

▼ Why removing the scale sharpens the row

Each row gets sharper. The widget uses $d_k = 16$, so the scaling divides every score by $\sqrt{16} = 4$. Turning it off multiplies every score of a row by 4, and with them every difference between two scores. The softmax responds only to these differences: for two keys j and l of the same row,

$$\frac{a_{ij}}{a_{il}} = e^{z_{ij} - z_{il}},$$

so a difference four times as large raises the ratio to its fourth power. More of the weight moves to the highest-scoring key, and the row concentrates on fewer keys. With the scaling on, the smaller scores give a flatter row. In the widget, `A` is the matrix of attention probabilities, and `Z` is the value blend H of formula (3), not the scores z_{ij}.

Back to the question

A2 · The MLP holds most of the block's weights

- Attention: `4D² = 1,048,576`
- MLP: `2·D·3D = 6D² = 1,572,864`
- MLP: 60% of the block's matrices

▼ Counting the block's parameters

For a general hidden width d_{ff}, the two MLP matrices hold $D\,d_{ff} + d_{ff}\,D = 2D\,d_{ff}$ parameters.

Attention holds the four projections W_Q, W_K, W_V and W_O, each $D \times D$ here, so $4D^2 = 1{,}048{,}576$ parameters at $D = 512$. The MLP with $d_{ff} = 3D$ holds $2 \cdot D \cdot 3D = 6D^2 = 1{,}572{,}864$. Of the block's $10D^2$ matrix parameters, $6D^2$ are in the MLP:

$$\frac{6D^2}{4D^2 + 6D^2} = \frac{6}{10} = 60\%.$$

Most of a block's weights are in its MLP.

The widget draws the same MLP. The three boxes are the widths $512 \to 1536 \to 512$, and the hidden box is three times as tall as the others because it is three times as wide. The curve compares GELU with ReLU. ReLU is zero for every negative input, while GELU is smooth and slightly negative for small negative inputs. The exercise implements the exact form $x\,\Phi(x)$.

Back to the question

A3 · At one block, the logits dominate memory

COST CALCULATOR

estimate first, measure second

d_model 512

n_layers 1

S (model length) 1,024

B (batch) 32

stored tensor dtype

fp32 | bf16 | fp16

FP32 master copy | materialized logits

Parameters (tied, exact)
6,817,280

Analytic memory total
872.02 MiB

Block matmuls / token
7,340,032 FLOPs

Tied output head / token
8,388,608 FLOPs

Untying adds 4.2M parameters. Values for more than one layer are extrapolations beyond Week 2's one-block model, toward Week 3's stacked model.

attention materialization

naive: 2 matrices | inspected: 1 matrix | fused: 0 full matrices

Analytic memory buckets



- Parameters 13.00 MiB
- Gradients 13.00 MiB
- Adam moments 52.01 MiB
- Master weights 26.01 MiB
- Width-linear activations 128.00 MiB
- Attention matrices 128.00 MiB
- Vocabulary logits 512.00 MiB

Every value here is an analytic prediction; none comes from an observed run. The estimate assumes a tied model without biases, d_ff = 3d, two RMSNorm gains per block plus a final gain, and kappa = 4 width-linear activations. It leaves out sharding, checkpointing, allocator overhead and backend workspaces.

- One block: logits 512 of 872 MiB
- Eight blocks: logits 512 MiB of 2.88 GiB
- Depth grows every bucket but the logits

▼ Reconcile the seven memory buckets

The calculator's controls describe one scenario. The default is a width of 512 (`d_model`), one layer, $S = 1024$, $B = 32$, BF16 stored tensors, naive attention with two full matrices, an FP32 master copy and materialized logits. The vocabulary is fixed at 8192 and the MLP width at $3D$. Change one control at a time, and predict before each change which readout will move.

The four readouts have different units: an exact tied parameter count, an analytic memory total in binary units, the block's matmul FLOPs per token, and the tied head's FLOPs per token. At the default they read 6,817,280, 872.02 MiB, 7,340,032 FLOPs and 8,388,608 FLOPs. The attention selector and the master and logit toggles change the analytic scenario. They do not report what a backend actually kept in memory.

The buckets at one block and at eight

The sums assume that all the listed tensors are live at the same time. The left column is the default. The right column sets the calculator to eight layers, the depth of the Week 3 exercise, with one head per block as in Week 2.

Bucket	Bytes	One block	Eight blocks
BF16 parameters	$2P$	13,634,560	50,349,056
BF16 gradients	$2P$	13,634,560	50,349,056
two FP32 Adam moments	$8P$	54,538,240	201,396,224
FP32 master copy	$4P$	27,269,120	100,698,112
activations, $\kappa = 4$	$4\,BSDL \cdot 2$	134,217,728	1,073,741,824
two BF16 attention matrices	$2\,BLS^2 \cdot 2$	134,217,728	1,073,741,824
BF16 logits	$BSV \cdot 2$	536,870,912	536,870,912
total		914,382,848	3,087,147,008
total in binary units		872.02 MiB = 0.85 GiB	2,944.13 MiB = 2.88 GiB

With $\kappa = 4$, the logits take 512 MiB at one block, more than the other six buckets combined. They stay the largest bucket for any κ below 16, since the activations take 32κ MiB here. The reason is $B \cdot S \cdot V$: with $V = 8192$, this one tensor outweighs everything that a single block stores. At eight blocks the logits are unchanged, because they do not depend on the depth, while every other bucket grows: the activations and the attention matrices eightfold, to 1 GiB each. The logits fall from 59% of the total to 17%, and every further block adds to the per-block buckets while the logits stay as they are.

What the toggles remove

Starting from the one-block default:

Toggled off	Total
nothing (the default)	872.02 MiB
materialized logits	360.02 MiB
full attention matrices (fused)	744.02 MiB
full attention matrices and the master copy	718.02 MiB
all three	206.02 MiB

A fused or chunked loss avoids materializing the full `[B,S,V]` logits.

What the estimate leaves out

A fused attention backend keeps no full `[B,S,S]` matrix, but its kernel still needs workspace. The estimate also leaves out fragmentation, saved tensors that the buckets do not list, and allocations

outside PyTorch. The FP32 master toggle does not depend on the stored dtype: for a pure-FP32 implementation without a separate master copy, leaving it on counts one FP32 copy of the weights twice.

Back to the question